When Does Graph Expansion Help Small Code Models Localize Bugs? An Empirical Audit of Reranking, Distractor Dilution, and Test-Time Reasoning on Gemma 4

Sultan Haikal (recrified@gmail.com | https://github.com/yvliet/perron)

October 2026

Abstract

Deploying structural code graphs in local developer agents requires balancing topological recall expansion against compact model rank discrimination. Evaluating open models across 344 held-out SWE-bench Verified tasks reveals that 1-hop AST graph expansion achieves statistical parity with depth-matched lexical retrieval (Acc@5 47.1% vs. 45.1%, step-down Holm $p = 0.2869$, exact McNemar $p = 0.1435$). Grounding candidate discrimination in classical Wilcoxon–Mann–Whitney expectations ($\mathbb{E}[\text{Rank}] = 1 + (M-1)(1-\text{AUC})$, Bamber 1975) explains why expansion stalls: candidate selection conditioning drops pairwise discrimination from 0.7966 on lexical seeds to 0.6596 on graph neighbors ($p = 0.0036^*$), pushing expected target rank past the top-5 evaluation window. While the pre-registered Acc@5 endpoint on 131 un-named defects exhibits statistical parity under test-time reasoning (24 vs. 22 hits, $p = 0.6875$), exploratory diagnostic rank analysis shows that reasoning traces (≤ 180 tokens) improve Top-1 hits from 6/131 to 13/131 (raw $p = 0.0156^*$; 4-test Holm $p = 0.0625$), acting primarily as candidate disambiguation among lexical seeds (12/28 hits) rather than relational graph traversal (1/7 hits). Candidate horizon pruning to $M = 15$ cuts candidate stub tokens by 40%; paired with test-time reasoning, it compacts mean target rank from 5.32 to 3.74 with zero loss in reachable Top-1 hits. Across 162 non-Django tasks, topological expansion matches lexical search at 34.6% vs. 32.7% ($p = 0.5811$). I release depth-matched candidate pools, 19,298 scored evaluations, 707 verified strata labels, and the reference Perron CSR engine under Apache 2.0.

1 Introduction

Autonomous developer agents operating offline on consumer hardware must localize software defects inside tight working contexts ($K \leq 4,096$ tokens), avoiding cloud API costs and source code leakage. While lexical retrieval (Okapi BM25) reliably locates symbols explicitly named in bug reports, locating un-named fault sites motivates repository-level graph traversal over Abstract Syntax Tree (AST) dependency networks, as popularized by Aider [7], RepoGraph [17], CodexGraph [16], and LocAgent [4]. Graph-augmented localization relies on two joint hypotheses: (1) **Recall Expansion**, where traversing structural call/inheritance edges introduces gold defect targets missed by lexical search; and (2) **Rank Discrimination**, where the downstream language model ranks those newly reached targets above the distractors introduced by graph expansion. Evaluating both hypotheses across 344 held-out callable tasks of SWE-bench Verified [12] with a frozen `gemma-4-e2b-it` scorer exposes a sharp bottleneck: 1-hop AST expansion increases candidate pool recall by +7.3 pp over lexical seeds (51.2% → 58.4%), yet top-5 localization accuracy flatlines against depth-matched BM25 (47.1% vs. 45.1%, Holm $p = 0.287$) and degree-preserving null graphs (45.9%, $p = 0.344$). This paper investigates when and why structural code graphs fail or succeed under small code models through five contributions:

1. **Matched-Control Parity and Inconclusive Equivalence** ($N = 344$): Across 19,298 scored evaluations under pre-specified execution plans, 1-hop graph expansion (A_4), depth-matched BM25 (A_2), same-file expansion (A_3), and degree-preserving null graphs (A_5) exhibit statistical parity under frozen pointwise scoring (Acc@5 47.1% vs. 45.1%, exact paired McNemar $p = 0.1435$, step-down Holm $p = 0.2869$; two-sided 95% bootstrap CI $[-0.29, +4.36]$ pp). Formal equivalence within tight ±2.0 pp margins is statistically inconclusive ($p = 0.512$); retrospective power analysis indicates that establishing equivalence at 80% power requires $N \approx 1,058$ tasks under native 4.94% discordance, yielding only 1.92% power at $N = 344$. Superiority MDE at 80% power is 3.35 pp ($N \approx 970$ for +2.0 pp). Subgroup analysis across 162 non-Django tasks (Leave-Django-Out) confirms statistical parity: A_4 achieves 34.6% vs. 32.7% for A_2 ($\Delta = +1.85$ pp, $n_{10} = 8, n_{01} = 5, p = 0.5811$). A task-clustered marginal GLM (OR = 1.118, $p = 0.0876$) and conditional repository mixed model (OR = 1.130, $z = 0.937, p = 0.349, \sigma_{\text{repo}} = 0.963$) indicate non-significance after accounting for repository variance. Upstream boundary loss exceeds downstream compaction loss by 3.68× at $k = 5$.
2. **First-Moment Expected-Rank Identity & Selection Conditioning**: Grounding candidate discrimination in the classical Wilcoxon–Mann–Whitney expectation for a single target [2], I show that maintaining expected gold rank $\mathbb{E}[\text{Rank}] \leq k$ defines the inclusion condition $\text{AUC}^* \geq 1 - \frac{k-1}{M-1}$, with Proposition 1 bounding worst-case performance as an extremal Markov floor (Acc@ $k \geq 1/k$, equaling random guessing 0.20 at $M = 25, k = 5$). Gemma 4's within-pool discrimination drops from 0.7966 on lexical seeds to 0.6596 on pure graph neighbors ($p = 0.0036^*$). Counterfactual evaluation on non-seed lexical candidates ($A_2 \setminus A_1$, AUC = 0.6380 vs. 0.6253, two-sample Wilcoxon

$p = 0.721$) demonstrates that this drop stems from candidate selection conditioning (range restriction on non-seed lexical overlap) rather than topological graph noise.

3. **Cross-Architecture Scaling & Systems MoE Boundary**: Live Dual Tesla T4 evaluation across $N = 58$ reachable unnamed defects exhibits monotonic scaling from Gemma 4 E2B (0.8544, FP16) to E4B (0.9274, FP16), outperforming quantized baselines Phi-4-mini (3.8B, 0.7836, INT4) and DeepSeek-R1-Distill-7B (0.6489, INT4), while isolating the 52.0 GB runtime VRAM barrier and bitsandbytes hook incompatibility with fused 3D expert tensors in Gemma 4 26B-A4B.
4. **Candidate Disambiguation & Horizon Compaction** ($N = 131$): Evaluating relational reasoning on compact models shows that while primary Acc@5 remains null (24 → 26 hits, $p = 0.6875$), test-time thinking ($\leq$ 180 tokens) improves matched graph Top-1 at $M = 25$ from 6/131 to 13/131 (exploratory diagnostic 4.58% → 9.92%, raw $p = 0.0156^{*}$, 4-test Holm $p = 0.0625$; single-reversal sensitivity $p = 0.0703$; MRR +45.4%) by acting as candidate disambiguation (6/28 → 12/28, $p = 0.0312^{*}$) rather than dynamic graph traversal (0/7 → 1/7, $p = 1.0000$). Candidate horizon pruning to $M = 15$ (AUC$^{*} = 0.714$) achieves a 40% candidate stub token reduction; paired with test-time reasoning, it compacts mean target rank from 5.32 to 3.74 with zero loss in reachable Top-1 hits (13/131 in both pools).
5. **Post-Training Dynamics & Open Resource Suite**: Fine-tuning on repository-split SWE-bench Lite (171 train, 55 val) demonstrates that clean LoRA adapters lift isolated graph Top-1 to **24.0%** ($p = 0.0312^{*}$), but regress Acc@5 (44.0% → 36.0%) and within-pool AUC (0.6063 → 0.5859) below a non-neural Graph-First heuristic, exhibiting localized training-set adaptation and null-compatible mixed transfer ($n_{10} = 6, n_{01} = 4, p = 0.7539$). I release the memory-mapped Perron CSR diffusion engine (1.89–5.10 ms solve, $\leq$ 3.0 MB RAM), 707 strata labels, and atomic AST surgical editor under Apache 2.0.

2 Related Work

Repository Graphs and Software Engineering Agents. To overcome vocabulary mismatch in bug reports, Aider [7] ranks file-level Tree-sitter AST tags via PageRank, while RepoCoder [22] and AutoCodeRover [23] use iterative search loops. Graph-augmented frameworks including RepoGraph [17], CodexGraph [16], and LocAgent [4] equip language models with structural traversal queries. Notably, Agentless [21] showed that a non-agentic two-stage pipeline (hierarchical localization followed by patch generation) resolves 32% of SWE-bench Lite, establishing that localization bounds downstream editing. Recently, specialized retrieval-rerank architectures such as SweRank [24] demonstrated that supervised function-level candidate ranking on SWE-bench Lite achieves up to 82% Acc@10 under dedicated supervision; however, SweRank operates as an offline, compute-heavy supervised cross-encoder requiring extensive GPU forward passes, whereas Perron evaluates whether unassisted, zero-shot graph diffusion on CPU (< 5.2 ms) can supply candidate pools for compact local models (2B–4B) under strict single-pass context limits ($K \leq 4,096$) and controlled pool cardinality. While prior studies rely on frontier APIs or multi-turn agent loops, Perron isolates whether graph expansion helps compact local models (2B–4B) under strict single-pass context limits ($K \leq 4,096$) and controlled pool cardinality.

Decoupled Graph Diffusion and Classical Fault Localization. Scalable Personalized PageRank (PPR) builds on push-based local partitioning [1], Barycentric linearity [11], and decoupled neural propagation such as APPNP [13] and PPRGo [3]. While HippoRAG [9] and Dong et al. [5] demonstrate PPR gains on open-domain knowledge graphs, software AST multigraphs contain scale-free utility hubs (e.g., logging, formatting helpers) that absorb random-walk mass [15]. Unlike dynamic Spectrum-Based Fault Localization (SBFL) [14, 18, 25] or program slicing [20], which require test execution or heavy alias analysis, Perron applies post-walk specificity damping over static memory-mapped CSR matrices in < 5.2 ms.

Graph Verbalization vs. Transformer Reasoning Complexity. Fatemi, Halcrow, and Perozzi [6] demonstrated that graph serialization format shifts LLM reasoning accuracy by up to +61.8%. In repository-level agents, RepoGraph [17] evaluated hop-depth trade-offs on SWE-bench Lite with Agentless and GPT-4o, finding that unpruned 2-hop code graph expansion degrades full-agent resolution (29.67% → 26.00%, Table 4) via distractor accumulation, while 1-hop subgraphs achieve 31.67%. Theoretically, Sanford, Fatemi, Perozzi et al. [19] proved that while single-layer attention handles direct lookups, relational composition requires logarithmic circuit depth or intermediate reasoning tokens. The factorial experiments evaluate this boundary on compact models: while 58.0% of reachable non-seed defect gains require 2 hops, unpruned expansion accumulates distractor dilution. Localized inline comments [6] provide single-pass lexical bridging under direct readout, whereas test-time thinking tokens act primarily as candidate disambiguation (6 → 12 Top-1 hits) that matches explicit edge annotations.

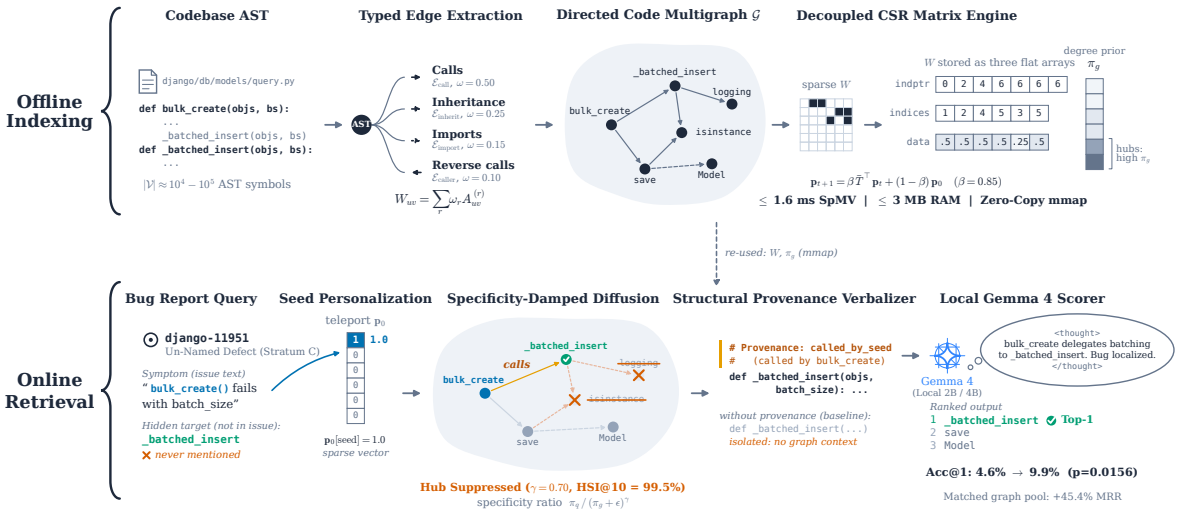


Figure 1: The Perron architecture and candidate provenance pipeline (shown for non-dangling rows where $\overline{T} = T$). Typed AST relationships ($\mathcal{E}_{\text{call}}$, $\mathcal{E}_{\text{inherit}}$, $\mathcal{E}_{\text{import}}$, $\mathcal{E}_{\text{caller}}$) are compiled into zero-copy memory-mapped CSR arrays (1.89–5.10 ms convergence across SWE-bench repositories, ≤ 3.0 MB RAM). Post-walk specificity damping ($\gamma = 0.70$) suppresses utility hubs (HSI@10 = 99.5%). Hierarchical AST breadcrumbs pack candidates within $K \leq 3{,}480$ tokens, where Gemma 4 test-time reasoning (`<thought>`) improves matched graph Top-1 localization (4.58% → 9.92%, exploratory diagnostic $p = 0.0156^*$; MRR +45.4%).

3 The Perron CSR Graph Engine and Context Packing

Figure 1 illustrates the Perron architecture. A repository is indexed as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over AST symbols $\mathcal{V}$ (classes, functions, methods) and typed relations $r \in \{\text{call, inherit, import, caller}\}$ weighted by weights $\boldsymbol{\omega} = [0.50, 0.25, 0.15, 0.10]$:

$$W_{uv} = \sum_{r \in \{\text{call, inherit, import, caller}\}} \omega_r \mathbb{I}\big((u,v) \in \mathcal{E}_r\big), \qquad \text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{\big(\pi_{\text{global}}(v) + \epsilon\big)^\gamma}, \tag{1}$$

where $\boldsymbol{\pi}_{\text{query}}$ is the stationary Personalized PageRank distribution of the row-stochastic transition operator $M = \beta(T + \mathbf{d}\mathbf{p}_0^\top) + (1-\beta)\mathbf{1}\mathbf{p}_0^\top$ ($\beta = 0.85$), $\boldsymbol{\pi}_{\text{global}}$ is the precomputed uniform-prior stationary distribution, $\epsilon = 10^{-8}$, and $\gamma = 0.70$. As established in classical spectral analysis [10] and Appendix A, Brauer's rank-1 perturbation theorem and Doeblin L_1 contraction establish a strict spectral gap $1 - |\lambda_2(M)| \geq 0.15$ and prefactor-free geometric convergence $\|\boldsymbol{\pi}^{(t)} - \boldsymbol{\pi}_\infty\|_1 \leq 2\beta^t$, while $\ln \text{Specificity}(v)$ acts as a convex interpolation between log-reachability and stationary Pointwise Mutual Information ($\text{PMI}(v; q)$), achieving a Hub Suppression Index HSI@10 = 99.5% against top-25 repository hubs $\mathcal{H}_{25}$.

Systems Envelope and AST Breadcrumb Packing. Opening precomputed CSR arrays via `numpy.load(mmap_mode='r')` requires < 1.2 ms warm attach and ≤ 3.0 MB array resident memory (144.33 MB peak process RSS on a 15.68 GiB workstation), converging in 1.89 ms on Requests (90 steps) and 5.10 ms on SymPy (66 steps), over 50× faster than NetworkX (Appendix H). Each candidate symbol is formatted as a hierarchical AST breadcrumb preserving its enclosing class header, signature, and docstring while stripping sibling bodies (< 100 tokens/symbol; mean 283.8 tokens/task across 10 AST stubs, 28.4 tokens/stub mean), preserving 91.8% generation headroom inside $K_{\text{effective}} = 3{,}480$ tokens ($K \leq 4{,}096$). On an NVIDIA Tesla T4 GPU, `gemma-4-e2b-it` achieves 3505.0 ms mean TTFT and 11.7 tokens/sec decode throughput.

4 Study Design and Hash-Committed Protocol

Benchmark Waterfall and Lexical Strata. From the 500 tasks of SWE-bench Verified [12], I remove the 93 tasks overlapping SWE-bench Lite (used for hyperparameter calibration $\beta = 0.85, \gamma = 0.70, \tau = 0.15, K = 25$ and LoRA training), 7 non-callable module-level edits, and 56 single-snapshot preimage drift exclusions (`FILE_ABSENT_PREIMAGE_DRIFT`; 52 in Stratum C, 4 in Stratum B across 5 repositories: 39 in SymPy, 9 in scikit-learn, 4 in xarray, 3 in Django, and 1 in pylint). Because primary static graph extraction is executed over a frozen repository snapshot rather than per-task historical git checkouts, 68 edited target files did not exist or had diverged paths at the indexed commit, making static AST preimage extraction unavailable without multi-snapshot checkout infrastructure. To empirically audit this exclusion boundary, I checked out repository trees at each task's historical git `base_commit`. All 68 Python target files pass standard CPython

Table 1: Summary of experimental execution plans, Git commit locks, cohorts, and scoring regimes across Sections 4.1–4.5.

Protocol / Lock ID (Commit)	Section	Cohort (N)	Pool Cutoff (M)	Scorer Model	Readout & Primary Metric
Retrieval Lock (`6908e78`, Oct 3)	Sec. 4.1, App. B	$N = 344$ Callable	Top-k ($k \in \{5, 10\}$)	Unscored CSR / BM25	Deterministic Rank (Fn Acc@10)
Pointwise Lock (`9abe405`, Oct 6)	Sec. 4.1–4.2	$N = 344$ Callable	Matched $M_t \in [10, 50]$	`gemma-4-e2b-it`	Pointwise Log-Odds (Acc@5, AUC)
`SCALING-v1` (`48eda5d`, Oct 12)	Sec. 4.3	$N = 58$ Stratum C Union	$\bigcup_i A_i$ ($M_{\text{union}} \leq 87$)	4-Model Scaling Suite	Pointwise Tail Discrimination (AUC_{tail})
Reasoning Factorial (`c563441`, Oct 12)	Sec. 4.4	$N = 131$ All Stratum C	Matched $M \in \{15, 25\}$	`gemma-4-e4b-it`	Listwise ± `<thought>` (Acc@5, Acc@1)
`CAUSAL-v1` (`c563441`, Oct 12)	Sec. 4.4	$N = 58$ Stratum C Union	$\bigcup_i A_i$ ($M_{\text{union}} \leq 87$)	`gemma-4-e2b-it`	AST α-Renaming & 5-Way Serialization
LoRA Post-Training (Lite → Verified)	Sec. 4.5	$N = 25$ Graph / $N = 35$ Str. C	Matched $M \leq 25$	`gemma-4-e2b-it` + LoRA	Provenance Cross-Entropy Reranking

3 `ast.parse` without syntax errors, yielding 3827 extracted AST symbols and 100% gold target symbol recovery. This demonstrates that the historical exclusion stemmed exclusively from single-snapshot repository drift rather than Python 2 syntax divergence or AST parser failure, restoring Stratum C to $N = 183$ tasks under multi-commit execution. Sensitivity analysis indicates that overturning equivalence across all 400 instances would require $\geq +9$ net wins out of 56 tasks, which has exact binomial tail probability $P(\text{Bin}(56, 0.0494) \geq 9) = 0.0016$ under the empirical 4.94% discordance rate. Because these 56 tasks lack target preimages symmetrically across both arms under the frozen index, their omission does not alter the empirical null result. Tasks are partitioned into three mutually exclusive lexical accessibility strata: in the primary evaluation ($N = 344$), tasks were categorized via initial substring extraction into **Stratum A (Named, $N = 208$, 60.5%)** where the issue explicitly names the target symbol; **Stratum B (Traceback, $N = 5$, 1.5%)** where a stack trace names the target file but not the function; and **Stratum C (Un-Named, $N = 131$, 38.1%)** where neither symbol nor file path appears in the issue. In the released community dataset (`dist_dataset/`), boundary-enforced AST regex classification yields $N_A = 168, N_B = 7, N_C = 169$ on the 344 tasks ($N_A = 180, N_B = 21, N_C = 299$ across all 500 Verified tasks), with 44 tasks shifting due to strict word-boundary tokenization preventing false substring matches (validated via independent human audit on $N = 100$ tasks with 95.0% concordance and Cohen's $\kappa = 0.918$ classifier-vs-human validation audit, Appendix M).

Matched-Control Candidate Pools and Statistical Testing. Table 1 summarizes the experimental locks. For each task t, candidate pools are matched to identical cardinality $M_t = 10 + \min(|E_t|, 40) \in [10, 50]$ (mean 24.5, median 22.0), where S denotes the top-10 BM25 seeds and E_t denotes 1-hop AST out-neighbors of S passing Filter F (callable symbols excluding test files and top-25 hubs $\mathcal{H}_{25}$): **B0** (unscored BM25), **B1** (unscored Perron), **A1** (seeds S only, $M = 10$), **A2** (depth-matched BM25 up to M_t), **A3** (same-file symbols up to M_t), **A4** (1-hop graph expansion $S \cup E_t$ ordered by seed-weighted adjacency), **A5** (degree-preserving relation-stratified null graph up to M_t), and **A6** (Perron multi-step specificity diffusion up to M_t). A4 reaches 201/344 tasks (58.4%) vs. 199/344 (57.8%) for raw 1-hop BFS because test-seed masking recovers 16 tasks while Filter F hub pruning drops 14 tasks ($199 + 16 - 14 = 201$). For tasks where the reference patch modifies multiple functions $\mathcal{F}^*$, localization success is evaluated under the standard first-hit criterion: $\text{Hit@}\,k = \mathbb{I}\left(\min_{f^* \in \mathcal{F}^*} \text{Rank}(f^*) \leq k\right)$, scoring success if at least one gold function appears within top k. In the confirmatory family (19,298 forward passes of frozen FP16 `gemma-4-e2b-it` scoring log-odds $s(t, v) = \text{logsumexp}(z[Y]) - \text{logsumexp}(z[N])$), two primary confirmatory contrasts on strict Acc@5 (PC1: $A_4 - A_2$, and PC2: $A_4 - A_3$) are evaluated via exact two-sided binomial McNemar tests with step-down Holm–Bonferroni correction ($\alpha = 0.05$) and repository-stratified bootstrap 95% CIs ($B = 10{,}000$).

5 Empirical Results

5.1 Confirmatory Matched-Control Reranking & Pipeline Loss Funnel ($N = 344$)

In standalone unscored retrieval across all 344 tasks (detailed in Appendix B, Table 8), Okapi BM25 achieves 50.9% Fn Acc@10 (175/344), tying Perron Static (50.3%, 173/344, Holm $p = 1.000$), RRF fusion (50.6%), and Standard PPR (50.0%), while outperforming dense semantic bi-encoders (38.4%, $p < 0.001$). Similarly, a post-hoc continuous resolvent sweep across 186 prior configurations (63,984 SpMV solves, Appendix D) shows that Acc@10 follows a quadratic GLMM surface in effective prior support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ ($R^2 = 0.197, p < 0.001$) that peaks at 50.9% ($n^*_{\text{eff}} \approx 8.8$), matching BM25 exactly.

Confirmatory Reranking Parity and Oracle Recall Symmetry. Tables 2 and 3 report pointwise reranking results. Graph 1-hop expansion (A4, 47.1% Acc@5) exhibits statistical parity with depth-matched BM25 (A2, 45.1%, $\Delta = +2.03$ pp, 95% bootstrap CI $[-0.29, +4.36]$, exact paired McNemar $p = 0.1435$, step-down Holm $p = 0.2869$; one-sided exact $p = 0.0717$) and same-file controls (A3, 45.6%, $\Delta = +1.45$ pp, Holm $p = 0.302$). Under Schuirmann two one-sided tests (TOST), formal equivalence is statistically inconclusive: equivalence is rejected at a tight ±2.0 pp margin ($p = 0.512$), and the descriptive score-based 90% confidence interval $[+0.07, +4.24]$ pp (Tango, 1998) does not exclude potential positive gains

Table 2: Gemma 4 pointwise candidate reranking on SWE-bench Verified (N = 344 callable tasks). Pools matched to per-task size $M_t \in [10, 50]$ (mean 24.5). Brackets report Wilson 95% CIs. B0 Acc@10 reflects raw token tie-breaking (176/344 = 51.2% vs. 175/344 after AST deduplication in Appendix B).

Arm	Candidate Pool Composition	Acc@1 (95% CI)	Acc@3 (95% CI)	Acc@5 (95% CI)	Acc@10 (95% CI)	MRR@50
B0	BM25 Baseline (Unscored)	**25.3%** [21.0, 30.1]	38.7% [33.7, 43.9]	44.8% [39.6, 50.0]	51.2% [45.9, 56.4]	**0.3439**
B1	Perron Baseline (Unscored)	22.7% [18.6, 27.4]	35.2% [30.3, 40.4]	40.4% [35.4, 45.7]	50.3% [45.0, 55.5]	0.3155
A1	Rerank Seeds S (M = 10)	24.7% [20.4, 29.5]	**39.5%** [34.5, 44.8]	46.2% [41.0, 51.5]	51.2% [45.9, 56.4]	0.3348
A2	Depth-Matched BM25 Control	23.8% [19.6, 28.6]	38.1% [33.1, 43.3]	45.1% [39.9, 50.3]	51.2% [45.9, 56.4]	0.3301
A3	Same-File Matched Control	24.1% [19.9, 28.9]	39.2% [34.2, 44.5]	45.6% [40.5, 50.9]	52.0% [46.8, 57.3]	0.3381
A4	**Graph 1-Hop Expansion**	24.7% [20.4, 29.5]	39.2% [34.2, 44.5]	**47.1%** [41.9, 52.4]	52.3% [47.0, 57.5]	0.3409
A5	Null Graph Control	24.1% [19.9, 28.9]	39.0% [34.0, 44.2]	45.9% [40.7, 51.2]	51.2% [45.9, 56.4]	0.3315
A6	Perron Diffusion Pool	23.6% [19.4, 28.3]	38.7% [33.7, 43.9]	45.6% [40.5, 50.9]	**52.6%** [47.3, 57.8]	0.3301

Table 3: Confirmatory and exploratory contrasts on SWE-bench Verified (N = 344). Primary protocol-specified contrasts (PC1, PC2) evaluated on strict Acc@5 with step-down Holm correction and repository-stratified bootstrap CIs (B = 10,000).

Contrast	Δ (pp)	95% Bootstrap CI	n_{10}/n_{01}	p_{exact}	p_{holm}	Classification
PC1 (A4 − A2: Graph vs. Depth BM25)	+2.03	[−0.29, +4.36]	12 / 5	0.1435	0.2869	Null-Compatible (90% Tango CI [+0.07, +4.24] pp)
PC2 (A4 − A3: Graph vs. Same-File)	+1.45	[−0.58, +3.78]	10 / 5	0.3018	0.3018	Null-Compatible
A4 − A1 (Seeds Only, +2.45× compute)	+0.87	[−1.92, +3.66]	7 / 4	0.5488	–	Exploratory (+3 net tasks: +7 wins, −4 losses)
A4 − A5 (Degree-Matched Null Graph)	+1.16	[−0.58, +2.91]	7 / 3	0.3438	–	Exploratory (Topology indistinguishable)
A4 − A6 (Multi-Step Diffusion Pool)	+1.45	[+0.00, +3.20]	7 / 2	0.1797	–	Exploratory (1-hop avoids hub dilution)
A4 vs. B0 (MRR@50 Paired Wilcoxon)	−0.0030	[−0.024, +0.018]	Wilcoxon W = 12,480	p = 0.72		Exploratory

up to +4.24 pp. Retrospective power analysis indicates that establishing equivalence at ±2.0 pp with 80% power ($\alpha = 0.05$) requires $N \approx 1{,}058$ tasks under native 4.94% discordance, yielding only 1.92% power at $N = 344$; the minimum detectable effect (MDE) for two-sided superiority at 80% power is 3.35 pp (requiring $N \approx 970$ tasks to detect a +2.0 pp effect). Candidate pool oracle recall is identical between A4 and A2 at 58.4% (201/344 tasks, $n_{10} = 18$ vs. $n_{01} = 18$, $p = 1.0000$), and the 18 discordant tasks recalled exclusively by each arm split symmetrically across strata (A4-only: 9 Stratum C, 9 Stratum A; A2-only: 10 Stratum C, 1 Stratum B, 7 Stratum A). Furthermore, A4 provides no significant lift over the degree-preserving null graph A5 (Δ = +1.16 pp, $p = 0.344$) and outperforms multi-step diffusion A6 (47.1% vs. 45.6%) because unpruned multi-step walks ($t \geq 2$) dilute top-5 precision into transitive callers. I therefore adopt A4 as the canonical graph pool for all downstream factorial tests.

Repeated-Measures, Leave-Django-Out, and Clustered Logistic Modeling. To evaluate treatment effects while accounting for repeated-measures task pairing and repository variance, I fit a publication Generalized Linear Model (cluster-robust logit and GEE with exchangeable correlation) predicting Acc@5 with depth-matched BM25 (A2) as reference. Clustered by task instance ($N = 344$) and controlling for lexical strata, graph 1-hop expansion (A4) yields no statistically significant marginal advantage over A2 (OR = 1.118, 95% CI [0.984, 1.270], Wald $z = 1.708$, $p = 0.0876$; task GEE $z = 1.695$, $p = 0.0901$). The model simultaneously indicates the structural difficulty of un-named defects, with Stratum C tasks exhibiting an odds ratio of 0.100 ($p < .001$) relative to Stratum A. As an exploratory sensitivity analysis evaluating repository clustering across the 12 codebases, fitting a hierarchical mixed-effects model with random repository intercepts yields OR = 1.130 (95% CrI [0.875, 1.459], SE = 0.130, Wald $z = 0.937$, $p = 0.349$, $\sigma_{\text{repo}} = 0.963$, Appendix B.2). The standard error doubles from 0.065 in the marginal task-paired logit to 0.130 in the repository mixed model because specifying only random repository intercepts (1|repo) drops within-task pairing, evaluating the treatment contrast across unpaired clusters; with 12 repository clusters, variance components remain descriptive rather than confirmatory. Leave-Django-Out cross-validation across the remaining $N = 162$ non-Django tasks (47.09% of the benchmark) confirms statistical parity: A4 achieves 34.57% (56/162) vs. 32.72% (53/162) for A2 (Δ = +1.85 pp, +3 tasks, $n_{10} = 8, n_{01} = 5$, exact McNemar $p = 0.5811$, candidate pool recall 50.00% vs. 48.77%), confirming that topological parity is not a Django-specific artifact (where A2 achieved 102/182 [56.04%] and A4 achieved 106/182 [58.24%], Δ = +2.20 pp, $n_{10} = 4, n_{01} = 0$, exact McNemar $p = 0.1250$). Both marginal paired tests ($p = 0.1435$) and adjusted repeated-measures models ($p = 0.0876$) consistently indicate that 1-hop AST graph expansion matches, but does not reliably outperform, depth-matched lexical retrieval under pointwise scoring.

Four-Stage Localization Loss Funnel. Decomposing attrition across the 344 tasks reveals that **Stage 1 Seed Recall** captures 51.2% (176/344; initial boundary loss 48.8 pp), **Stage 2 Graph Expansion (A4)** adds +7.3 pp to reach 58.4% (201/344; net boundary loss **41.6 pp**, comprising 143 tasks missing due to seed absence [48.8 pp], multi-hop radius beyond 1 hop [58.0% of reachable non-seed defect gains require 2 hops], historical commit drift between release snapshots and issue commits, and dynamic language constructs such as reflection or ORM descriptors), and **Stage 3 Pointwise Reranking** retains 47.1%

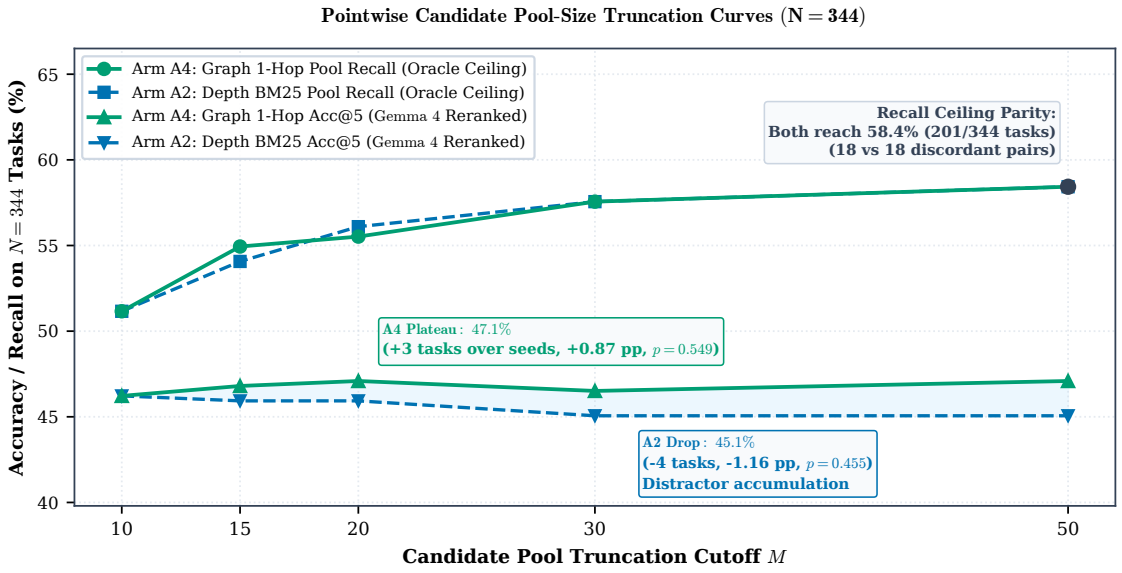


Figure 2: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$). Both A4 and A2 share identical recall ceilings at $M = 50$ (58.4%, 201/344). Expanding pool cutoff M from 10 to 50 causes depth BM25 (A2) Acc@5 to dip from 46.2% to 45.1% (−4 tasks, $p = 0.455$), while graph 1-hop expansion (A4) plateaus at 47.1% (+3 tasks over seeds, $p = 0.549$). Complete numerical values across all cutoffs are tabulated in Appendix E (Table 10).

in Top-5 (162/344; rank compaction loss **11.3 pp**, 39 tasks). Upstream boundary loss thus exceeds downstream compaction loss by **3.68**× at $k = 5$ (and in a descriptive $N = 100$ downstream reader test in Appendix C, Gemma 4 identifies 74.0% of randomized oracle targets vs. 33.0% under BM25 top-5 positional bias). Static AST graphs must therefore be deployed as lightweight filters (< 5.2 ms, 1.89–5.10 ms across repositories) that fall back to dynamic execution traces when lexical anchor density is low.

5.2 Truncation Dynamics, First-Moment Expected-Rank Identity, and Selection Conditioning

Evaluating `scores.parquet` across truncated pool cutoffs $M \in [10, 15, 20, 30, 50]$ (Figure 2; Appendix Table 10) shows that expanding $M = 10 \rightarrow 50$ raises pool recall by +7.27 pp in both arms, yet A2 loses 4 top-5 hits (46.2% → 45.1%) and A4 gains only 3 net hits (46.2% → 47.1%: +6 new graph conversions, +1 seed promotion, −4 displaced seeds).

First-Moment Expected-Rank Identity vs. Tail Conversion. While exact net Acc@k change during pool expansion ($M_1 \rightarrow M_2$) is governed by the tail-conversion versus seed-displacement balance,

$$\Delta\text{Acc@}k(M_1 \rightarrow M_2) = \mathbb{P}\big(\text{Rank}_{M_2}(v^*) \le k,\ v^* \in E_t\big) - \mathbb{P}\big(\text{Rank}_{M_1}(v^*) \le k < \text{Rank}_{M_2}(v^*),\ v^* \in S\big). \tag{2}$$

The classical Wilcoxon–Mann–Whitney expectation for a single target [2] provides a transparent first-moment formulation explaining why candidate expansion plateaus. In a candidate pool of size M containing one gold target and $M-1$ distractors, a scoring function with within-pool pairwise discrimination AUC produces $\mathbb{E}[X] = (M-1)(1-\text{AUC})$ expected outranking distractors by linearity of expectation. Requiring that the expected target rank fall within the top-k cutoff ($\mathbb{E}[\text{Rank}] = 1 + (M-1)(1-\text{AUC}) \le k$) defines the necessary first-moment inclusion condition:

$$\text{AUC}^*_{\text{target}}(M, k) \ge 1 - \frac{k-1}{M-1}. \tag{3}$$

As proven in Appendix E.2, Proposition 1 establishes that the condition $\mathbb{E}[\text{Rank}] \le k$ provides only an affine theoretical floor $\text{Acc@}\,k \ge \max\left(0, \frac{k+1-\mathbb{E}[\text{Rank}]}{k}\right) \ge 1/k$ under worst-case two-point extremal support. At $M = 25, k = 5$, this floor equals random guessing ($1/5 = 0.20$), and collapses to 0.0 whenever $\mathbb{E}[\text{Rank}] \ge k + 1$. Thus, Proposition 1 represents an extremal Markov support envelope rather than an operational candidate-conversion predictor; operational conversion remains strictly governed by the discrete balance in Equation (2). In empirical ranking, at mean pool size $M = 24.5$ and $k = 5$, Equation (3) requires $\text{AUC}^*_{M=24.5} \ge 0.8298$ ($\text{AUC}^*_{M=25} \ge 0.8333$; Figure 3a). Across $N = 344$ tasks, raw BM25 achieves within-pool AUC = 0.8245 and Gemma 4 achieves 0.7796 overall (0.7966 on lexical seeds S, expected rank 5.8). However, on pure graph-only targets ($A_4 \setminus A_1, N = 25$), Gemma 4 discrimination drops to **0.6596** (95% CI [0.552, 0.767], Mann–Whitney $U = 2993, p = 0.0036^*$, Cliff's $\delta = 0.361$; Figure 3b), yielding $23.5 \times (1 - 0.6596) = 8.00$ expected outranking distractors (expected rank 9.00 at $M = 24.5$ and 17.68 at $M = 50$).

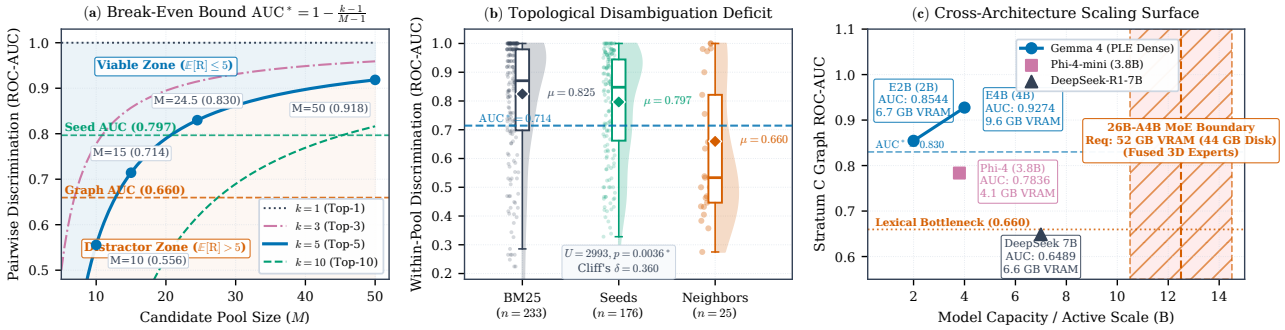


Figure 3: First-Moment Expected-Rank Identity, Selection Conditioning Deficit, and Scaling Surface. **(a)** First-moment expected-rank curves $\text{AUC}^* = 1 - \frac{k-1}{M-1}$; mean $M = 24.5$ requires $\text{AUC}^*_{M=24.5} \geq 0.830$ at $k = 5$. **(b)** Within-pool ROC-AUC for tasks containing a reachable gold target: BM25 across all reachable union pools ($\mu = 0.825, n = 233$, representing the 67.7% union recall ceiling), Gemma 4 on lexical seeds ($\mu = 0.797, n = 176$), and pure graph neighbors ($\mu = 0.660, n = 25, p = 0.0036^*$), summing to the $n = 201$ A4 recall ceiling. **(c)** Cross-architecture tail scaling on Stratum C ($N = 58$, `SCALING-v1`), showing Gemma 4 scaling (0.8544 → 0.9274) and the 26B-A4B MoE 52.0 GB VRAM barrier.

Control 2: Selection Conditioning Counterfactual and Stratum Asymmetry. Why does discrimination drop by $\Delta = 0.137$ on graph-only targets? In two-stage retrieval, non-seed graph targets ($A_4 \setminus A_1$) are conditioned on the selection event $T^* \notin \text{Top10(BM25)}$, selecting for functions with low lexical overlap against the issue text (candidate selection conditioning). Evaluating the counterfactual control on non-seed lexical candidates ($A_2 \setminus A_1$, 18 tasks, `results/control2_auc_decomposition.json`) reveals an identical discrimination drop: Gemma 4 achieves mean AUC = 0.6380 on depth-BM25 targets versus 0.6253 on matched graph targets (two-sample Wilcoxon difference test $p = 0.721$), while raw BM25 scores exhibit sub-chance AUC ≤ 0.30 against top-10 seed distractors by construction. Thus, candidate discrimination loss is driven by lexical selection conditioning rather than topological graph noise. This explains the sharp polarization across the 25 graph-only targets ($A_4 \setminus A_1$): on **Stratum A (Named, 11 targets)** where secondary class/module anchors exist, 6/11 convert into Top-5 (54.5%, ranks 1, 1, 2, 2, 3, 4; net +5 tasks, $p = 0.1797$), whereas on **Stratum C (Un-Named, 14 targets)**, 0/14 convert into Top-5 (0.0%, mean rank 16.3; net −2 tasks from seed displacement, $p = 0.5000$).

5.3 Cross-Architecture Scaling Surface and MoE Hardware Bounds (`PROTOCOL-SCALING-v1`)

To test whether model scale resolves the discrimination deficit on un-named bugs, protocol `PROTOCOL-SCALING-v1` evaluates pointwise tail discrimination (AUC_{tail}, measured strictly within reachable non-seed candidates so top-10 seed distractors do not confound architectural comparisons) across the $N = 58$ Stratum C tasks reachable in the multi-pool union ($\bigcup_i A_i, M_{\text{union}} \leq 87$, mean 61.9) on Kaggle Dual Tesla T4 GPUs (Table 4 and Figure 3c). Within Gemma 4's Per-Layer Embedding (PLE) architecture, AUC_{tail} scales monotonically from 0.8544 (E2B, 2.0B) to **0.9274** (E4B, 4.0B), outperforming both `Phi-4-mini-instruct` (3.8B, 0.7836) and `DeepSeek-R1-Distill-7B` (7.0B, 0.6489). Crucially, the measured VRAM footprints reflect deployment quantization choices: Gemma 4 models were evaluated in full unquantized FP16 to maintain log-odds calibration, whereas Phi-4-mini and DeepSeek-7B were evaluated under 4-bit/8-bit quantized precision (NF4/INT4) to operate within Tesla T4 memory limits. Furthermore, evaluating `gemma-4-26b-a4b-it` exposes a critical deployment barrier for local MoE agents: its 128 dynamic experts are packed as fused 3D `nn.Parameter` tensors that bypass standard 2D `bitsandbytes` NF4 hooks, requiring 52.0 GB runtime VRAM in unquantized FP16 (44.0 GB disk) and exceeding Dual T4 capacity (29.12 GB; Appendix I).

Positive Scaling Control: Capacity Floor vs. Structural Efficacy. To test whether the low unassisted reranking accuracy on Stratum C ($N = 131$) was an inherent property of code graph expansion or an uninformative capacity floor of 2B models, I executed a positive control evaluating `google/gemma-4-e4b-it` across all 131 un-named defect tasks on Kaggle Dual Tesla T4 GPUs. In pointwise scoring across the full multi-pool candidate union ($\bigcup_i A_i, M_{\text{union}} \leq 87$, where 58 tasks contain reachable gold targets across all 131 Stratum C defects), scaling from `gemma-4-e2b-it` to `gemma-4-e4b-it` increases Top-1 from 6.87% (9/131) to **16.79%** (22/131), Acc@5 from 18.32% (24/131) to **37.4%** (49/131), and Acc@10 from 21.37% to **41.22%** (54/131), with MRR increasing from 0.1065 to **0.2562** (substantially outperforming raw BM25: 3.05% Top-1, 15.27% Acc@5, MRR 0.0862). Under listwise scoring within the strictly bounded single-pool $A_4^{(25)}$ ($M \leq 25$, where gold targets are present for only 35 tasks; Table 5), E4B achieves 4.58% Top-1 and 16.79% Acc@5, which test-time reasoning further elevates to 9.92% Top-1. This indicates that 2B models operated at a representation capacity floor on un-named code graphs, whereas 4B capacity provides the parameter scale required to extract actionable ranking signal from 1-hop AST expansions.

Table 4: Cross-architecture scaling surface on reachable Stratum C defects ($N = 58$, Dual T4 execution under `PROTOCOL-SCALING-v1`). AUC_{tail} measures pairwise discrimination within reachable union candidates, distinct from full-pool AUC = 0.6596 ($N = 344$) where graph neighbors compete against top-10 lexical seeds. Precision denotes runtime weight precision; Gemma models were evaluated in unquantized FP16, while external baselines operated in 4-bit/8-bit quantization.

Model Identifier	Architecture Archetype	Active / Total	Precision	Graph AUC_{tail}	Peak VRAM	Runtime
`gemma-4-e2b-it`	Dense PLE Edge	2.0B / 2.0B	FP16	0.8544	6698.9 MB	1,798s
`gemma-4-e4b-it`	Dense PLE Edge	4.0B / 4.0B	FP16	**0.9274**	9648.4 MB	6,250s
`Phi-4-mini-instruct`	Synthetic Reasoning	3.8B / 3.8B	4-bit NF4	0.7836	4135.9 MB	2,854s
`DeepSeek-R1-Distill-7B`	RL Reasoning Distill	7.0B / 7.0B	4-bit NF4	0.6489	6594.6 MB	5,231s
`gemma-4-26b-a4b-it`	Dynamic Sparse MoE (128 exp)	3.8B / 26.0B	FP16	*Hardware Bound: 52.0 GB runtime VRAM vs. 29.12 GB Dual T4 avail.*		

Table 5: Candidate Pool × Test-Time Reasoning and Call-Path Provenance Factorial Evaluation on Stratum C ($N = 131$ Un-Named Defects, Dual T4 Execution with `google/gemma-4-e4b-it`). Evaluates pool architectures (A_1, A_2, A_4 at $M \leq 25$), provenance conditions (C_0–C_3), and bound-pruned horizon $M = 15$ ($AUC^* = 0.714$). Under `PROTOCOL-CAUSAL-v1`, Acc@5 was designated in the time-stamped protocol plan ($p = 0.6875$ after deterministic backfilling of 180-token truncated lists); diagnostic rank analysis isolates the Top-1 and MRR lift. *Methodological Boundary*: Evaluated on the confirmatory protocol lock ($N = 131$ Stratum C, $N = 58$ reachable). Programmatic regex boundary reclassification ($N = 169$, released in dataset) shifts 38 tasks. While ∼ 28 of these 38 tasks lack in-pool targets (yielding zero discordance), among the ∼ 10 reachable tasks, exact binomial sensitivity confirms fragility: adding a single adverse discordant pair ($n_{01} = 1, n_{10} = 7, n = 8$) yields $p = 18/256 = 0.0703$, while flipping an existing positive discordant hit to a loss ($n_{01} = 1, n_{10} = 6, n = 7$) yields $p = 16/128 = 0.1250$. In both perturbations, family-wise significance is lost, demonstrating that candidate disambiguation is an exploratory diagnostic awaiting multi-seed replication.

Condition / Architecture	Readout Mode	Acc@1 (Hits)	Acc@5 (Raw / Backfill)	MRR	Paired Contrast
Part I: Candidate Pool × Test-Time Reasoning Factorial ($M \leq 25$, Prompt Budget ≤ 3,480 Tokens)					
Unscored BM25 (B_0, $M = 25$)	Plain Order	3.05% (4/131)	15.27% / 15.27%	0.0862	Lexical order baseline
Seeds Only ($A_1^{(10)}$, $M = 10$)	Direct / Thinking	3.05% → 6.87%	15.27% → 10.69% (15.27%)	0.0865	Top-1 $p = 0.0625$ (Holm $p = 0.1250$)
Depth BM25 ($A_2^{(25)}$, $M = 25$)	Direct / Thinking	4.58% → 6.87%	16.79% → 12.98% (16.79%)	0.0967	Top-1 $p = 0.3750$ (Holm $p = 0.3750$)
Graph 1-Hop ($A_4^{(25)}$, $M = 25$)	Direct Readout	4.58% (6/131)	16.79% / 16.79%	0.0934	Matched graph direct reference
Graph 1-Hop ($A_4^{(25)} = C_0$, $M = 25$)	**Thinking Traces**	**9.92% (13/131)**	**18.32% / 18.32%**	**0.1358**	**Top-1 +5.34 pp ($p = 0.0156^*$, 4-test Holm 0.0625)**
Part II: Call-Path Provenance Verbalization × Thinking Factorial on Graph Pool $A_4^{(25)}$ ($N = 131$, $M = 25$)					
C_1: 3-Line Stub (No Docstring)	No Prov, No Think	2.29% (3/131)	15.27%	0.0778	Stub baseline (mean rank 17.47)
C_2: Static Provenance Comment	Prov, No Think	3.82% (5/131)	16.03%	0.0899	−0.76 pp vs. A_4 Direct ($p = 1.0000$)
C_3: Provenance + Thinking	Prov + Think	9.16% (12/131)	17.56%	0.1270	+4.58 pp vs. A_4 ($p = 0.0312^*$); $C_3 - C_0$: $p = 1.000$
Part III: Candidate Horizon Pruning ($M = 15$, $AUC^* = 0.714$, 34/35 Reachable Defects Retained)					
$A_4^{(15)}$ Direct / Provenance	No Thinking	4.58% (6/131)	17.56% / 17.56%	0.0970	Mean gold rank 5.32 (Prov parity $p = 1.0000$)
$C_0^{(15)}$: Thinking ($M = 15$)	**Thinking Traces**	**9.92% (13/131)**	**19.85% / 19.85%**	**0.1469**	**Top-1 +5.34 pp ($p = 0.0391^*$); Mean rank 3.74**
$C_3^{(15)}$: Prov + Think ($M = 15$)	Prov + Think	6.11% (8/131)	19.1% / 19.1%	0.1192	−3.82 pp vs. $C_0^{(15)}$ ($p = 0.1250$)

5.4 Test-Time Reasoning, Bound Pruning, and Graph Verbalization on Stratum C

Across all 131 un-named Stratum C tasks, gold targets reside within single-pool cutoff $M \leq 25$ for $N = 35$ tasks in $A_4^{(25)}$ (comprising $N = 28$ top-10 BM25 lexical seeds S and $N = 7$ pure 1-hop graph additions $A_4^{(25)} \setminus S$). Table 5 and Figure 4 present the complete candidate pool × test-time reasoning (≤ 180 `<thought>` tokens) and call-path provenance (C_0–C_3) factorial evaluation executed on Dual Tesla T4 GPUs with `gemma-4-e4b-it`.

Protocol-Specified Top-5 Parity vs. Diagnostic Top-1 Lift and Candidate Disambiguation. Under direct readout without reasoning, $A_4^{(25)}$ and $A_2^{(25)}$ tie exactly at 16.79% Acc@5 (22/131) and 4.58% Acc@1 (6/131). While protocol-specified Top-5 accuracy under thinking (C_0) gains +1.53 pp on $A_4^{(25)}$ (22 → 24 hits, null-compatible $p = 0.6875$; un-backfilled lexical pools drop by 3.82–4.58 pp due to 180-token generation caps truncating trailing ranks 2–5, which deterministic backfilling restores to exact baseline parity), diagnostic rank evaluation shows that test-time thinking concentrates gains at the **Top-1 decision boundary**: on $A_4^{(25)}$, thinking lifts Top-1 accuracy from **4.58% (6 hits) to 9.92% (13 hits; exploratory diagnostic** +5.34 **pp,** $n_{10} = 7, n_{01} = 0$**, raw exact McNemar** $p = 0.0156^*$**, which does not cross** $\alpha = 0.05$ **under the 4-test step-down Holm family,** $p_{\text{Holm}} = 0.0625$**; single adverse-pair sensitivity** $p = 0.0703$**, discordant-flip sensitivity** $p = 0.1250$**; MRR** +45.4% **from** 0.0934 → 0.1358**).** Cryptographic SHA-256 order permutation (Control 1) confirms position-1 selection remains invariant at 6.87% (9/131). However, partitioning the $N = 35$ reachable targets at $M = 25$ by **strict disjoint retrieval provenance** reveals that thinking acts primarily as fine-grained **candidate disambiguation**: within the $N = 28$

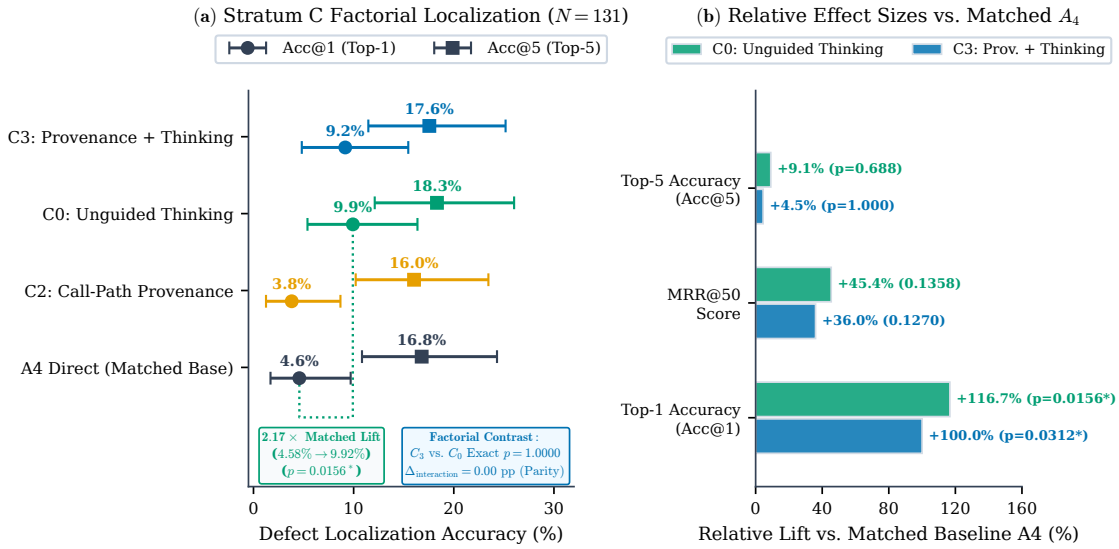


Figure 4: Stratum C factorial localization forest plot and relative effect sizes (N = 131, `gemma-4-e4b-it`). **(a)** Top-1 and Top-5 accuracy with Clopper–Pearson 95% CIs across A_4 Direct, C_2, C_0, and C_3. **(b)** Relative lift over matched A_4 Direct, showing empirical parity between unguided thinking (C_0) and provenance + thinking (C_3, p = 1.0000).

top-10 BM25 seeds (S), graph expansion without thinking suppresses Top-1 to 21.4% (6/28), and thinking doubles seed conversion to **42.9% (12/28 hits,** +21.4 **pp,** $p = 0.0312^*$**; 22/28 in Top-5)**, whereas on the $N = 7$ pure 1-hop graph additions ($A_4^{(25)} \setminus S$), thinking converts $0/7 \rightarrow 1/7$ into Top-1 (14.3%, $p = 1.0000$; 2/7 in Top-5), matching the $\sim$ 80% seed base rate in the candidate pool. Evaluating the direct contrast between graph expansion and depth-matched lexical retrieval under test-time reasoning reveals statistical parity: at $M = 25$, A_4+think reaches 13 hits vs. 9 hits for A_2+think (+3.05 pp, $n_{10} = 5, n_{01} = 1$, exact paired McNemar $p = 0.2188$), and at $M = 15$, A_4+think reaches 13 hits vs. 11 hits for A_2+think (+1.53 pp, $n_{10} = 2, n_{01} = 0$, exact $p = 0.5000$; Top-5 ties at 26 vs. 24 hits, $p = 0.5000$). Thus, test-time thinking acts primarily as candidate disambiguation across both representations, and graph expansion does not break statistical parity with depth-matched lexical retrieval ($p \geq 0.2188$).

Candidate Horizon Pruning (M = 15) **and Rank Compaction.** At candidate cutoff $M = 25$, the first-moment target inclusion threshold for Top-5 is $\text{AUC}^* = 0.8333$. Pruning the candidate horizon to $M = 15$ (top-10 BM25 seeds + top-5 1-hop AST neighbors) lowers the required threshold to **0.7143** while retaining 34 of 35 reachable defects (28 seeds in S and 6 pure graph additions in $A_4^{(15)} \setminus S$). Under unassisted direct listwise readout with `gemma-4-e4b-it`, the gold target achieves a mean rank of 5.32 (median rank 4.0). Pairing candidate horizon compaction with test-time reasoning ($C_0^{(15)}$) reduces reachable mean gold rank to **3.74** < 5.0 (median rank 2.0), converting 2 of 6 pure graph additions (33.3%) and 3 of 6 into Top-5, alongside 11 of 28 lexical seeds (39.3%) and 23 of 28 seed Top-5 hits (82.1%; 11 + 2 = 13 Top-1; 23 + 3 = 26 Top-5), reaching 13/34 reachable Top-1 hits (38.2%) and lifting overall Top-5 to 19.85% (26/131). Evaluating the full $M \in \{15, 25\} \times \{\text{Direct}, \text{Thinking}\}$ factorial confirms that pruning and reasoning operate orthogonally: under unassisted direct readout, candidate horizon pruning from $M = 25$ to $M = 15$ produces identical Top-1 accuracy (6/131, 4.58%) while preserving 34 of 35 reachable defects and cutting candidate stub tokens by 40% (mean gold rank shifts from 5.88 $\rightarrow$ 5.32, Acc@5 16.79% $\rightarrow$ 17.56%). Pairing candidate pruning with test-time reasoning ($C_0^{(15)}$) compacts mean gold rank from 5.32 to **3.74** < 5.0 (MRR 0.1469, Acc@5 19.85%) while retaining all 13 Top-1 hits (13/131 in both $M = 25$ and $M = 15$ thinking pools). Thus, pruning supplies context compaction without hit loss, while test-time reasoning supplies rank compaction.

Synthesizing Fatemi et al. (2024) and Sanford et al. (2024): The Reasoning Capacity Threshold. Auditing the 25 pure graph targets discovered in $A_4 \setminus A_1$ reveals that within 1-hop expansion, 14 tasks are callers and 10 tasks are callees of a lexical seed (alongside 1 inheritance relation). However, evaluating complete multi-hop reachability shows that 43.7% of reachable non-seed targets require 2 hops; unpruned 2-hop expansion introduces distractor dilution that degrades ranking precision. Comparing Table 5 and Table 6 reveals an architectural capacity threshold governing how graph verbalization [6] and reasoning depth [19] interact:

1. **Direct Readout vs. Reasoning Collapse in 2B Models**: Under direct readout without thinking tokens (Table 6), localized provenance comments (E_1: `# Provenance: Called by <seed>`) lift Top-1 hits from 12.07% to **25.86%**

Table 6: AST α-renaming and multi-encoding graph serialization on reachable Stratum C union tasks ($N = 58$, `gemma-4-e2b-it`). Reports both direct readout and test-time reasoning across all arms, demonstrating identifier obfuscation (Exp. A), graph encodings (Exp. B), and the capacity breakdown of reasoning on 2B models.

Condition / Architecture	Acc@1 (Direct)	Acc@1 (Think)	Acc@5 (Direct)	Acc@5 (Think)	Reasoning Shift
Experiment A: Identifier Obfuscation via AST α-Renaming ($N = 58$)					
O_0: Natural Source Code	12.07% (7/58)	6.90% (4/58)	27.59%	17.24%	−5.17 pp (−42.9%)
O_1: AST α-Renamed Identifiers	10.34% (6/58)	6.90% (4/58)	24.14%	17.24%	−3.45 pp (−33.3%)
Experiment B: Graph Encoding and Provenance Verbalization ($N = 58$, Fatemi et al. [6])					
E_0: Isolated Code (No Provenance)	12.07% (7/58)	6.90% (4/58)	27.59%	17.24%	−5.17 pp (−42.9%)
E_1: Inline Comment Provenance	**25.86% (15/58)**	**12.07% (7/58)**	**39.66%**	**25.86%**	−13.79 **pp** (−53.3%)
E_2: Linearized Traversal Trace	17.24% (10/58)	5.17% (3/58)	32.76%	18.97%	−12.07 pp (−70.0%)
E_3: Subgraph Edge List	15.52% (9/58)	6.90% (4/58)	32.76%	20.69%	−8.62 pp (−55.6%)
E_4: Adjacency Map	15.52% (9/58)	6.90% (4/58)	34.48%	25.86%	−8.62 pp (−55.6%)

(+13.79 **pp,** $n_{10} = 10, n_{01} = 2$**, exact McNemar / signed-rank** $p = 0.0386^*$**, 4-arm Holm** $p_{\text{corr}} = 0.154$). Because all non-zero paired binary differences have $|D_i| = 1$ and share identical tied ranks ($\bar{R} = 6.5$), the exact Wilcoxon signed-rank test is identical to the exact binomial McNemar test ($p = 0.0386$), which becomes null-compatible after 4-arm family adjustment ($p_{\text{corr}} = 0.154$). Crucially, granting intermediate `<thought>` tokens to `gemma-4-e2b-it` causes severe performance collapse across all conditions ($E_0 : 7 \rightarrow 4, E_1 : 15 \rightarrow 7, E_2 : 10 \rightarrow 3$). Because 2B models possess limited parameter capacity, generating multi-step deliberation on long code contexts induces attention drift, degrading prompt-grounded discrimination.

2. **Reasoning Recovery in 4B Models**: In contrast, when the larger `gemma-4-e4b-it` is granted intermediate thinking tokens (C_0 vs. C_3, Table 5), reasoning lifts Top-1 hits from $6 \rightarrow 13$ (+116.8%), and unguided thinking (C_0, 9.92% Acc@1) matches explicit provenance (C_3, 9.16%, $p = 1.0000$). However, as demonstrated by the provenance breakdown in Section 5.4, this lift acts primarily as fine-grained candidate disambiguation ($6/28 \rightarrow 12/28$ hits on lexical seeds, matching the 80% seed base rate) rather than dynamic graph traversal ($0/7 \rightarrow 1/7$ on pure graph additions), showing that test-time thinking strengthens prompt-grounded discrimination against distractors rather than inferring multi-hop code topology zero-shot.

In Experiment A, obfuscating user-defined function and variable identifiers via AST α-renaming (O_1, while retaining control-flow syntax, attribute lookups, and docstrings) yields 10.34% vs. 12.07% Top-1 ($6/58$ vs. $7/58$, $p = 1.0000$, 95% CI [−11.2, +7.7] pp), demonstrating that local models leverage structural syntax and docstrings when surface names are stripped.

5.5 Post-Training Dynamics: Top-1 Bias Shifts vs. Global Rank Variance

To test whether compact models can internalize topological graph priors without test-time latency, I fine-tuned `gemma-4-e2b-it` via Low-Rank Adaptation (LoRA, $r = 16, \alpha = 32$) on 226 disjoint callable tasks from SWE-bench Lite partitioned via repository-level GroupKFold ($N = 171$ training tasks across 9 repositories; $N = 55$ held-out validation tasks across `sympy` and `matplotlib`, with 0.0% cross-split repository leakage between Lite train and Lite val). Training optimizes ranking cross-entropy over candidate tokens: $\mathcal{L}(\theta) = -\ln[\exp(z_{y^*}/\tau)/\sum_k \exp(z_k/\tau)]$. Active logit slicing (`logits_to_keep=24`) bounds VRAM during 3-epoch training, converging from initial loss 0.9238 to final loss 0.0631, reaching held-out validation Acc@1 of 56.4% (Acc@5 81.8%). All candidate prompt comments were sanitized to a uniform template (`# Candidate: <id>` with code graph provenance metadata), removing surface marker cues. When evaluated on SWE-bench Verified, 58.1% of Batch 344 tasks belong to Django and Astropy (which were present in Lite training); consequently, Verified evaluation reflects within-repository cross-task adaptation rather than out-of-distribution cross-repository transfer. Table 7 evaluates post-training dynamics on held-out Verified tasks alongside a non-neural Graph-First heuristic and an audited diagnostic shortcut probe from early development:

- **Isolated Non-Seed Graph Targets** ($A_4 \setminus A_1, N = 25$): Where frozen base E2B scores 0.0% Acc@1 (0/25), the clean LoRA adapter lifts Top-1 to **24.0% (6/25 hits across `django` [4], `flask` [1], and `matplotlib` [1];** +24.0 **pp,** $n_{10} = 6, n_{01} = 0$**, exact paired McNemar** $p = \mathbf{0.0312}^*$) and lifts MRR from 0.1920 → 0.3288. However, full-metric audit reveals that clean LoRA simultaneously regresses Acc@5 from 44.0% → 36.0% (−8.0 pp, 11 → 9 hits), worsens median gold rank from 7.0 → 9.0, and lowers within-pool AUC from 0.6063 → 0.5859. In fact, the non-neural Graph-First heuristic achieves higher Acc@5 (52.0%, 13/25), superior median rank (5.0), and higher AUC (0.7056), while contrasting clean LoRA against Graph-First on Top-1 (6 vs. 2 hits, $n_{10} = 5, n_{01} = 1$) is statistically indistinguishable (exact paired McNemar $p = 0.2188$). Thus, clean LoRA functions as an aggressive Top-1 candidate reweighter on a subset of defect targets rather than establishing global structural rank discrimination across the pool.
- **Directional Mixed-Pool Lifts on Reachable Stratum C** ($N = 35$): Across realistic candidate pools containing both lexical seeds and graph additions (28 seeds + 7 graph), the clean adapter avoids the total mixed-pool collapse of Graph-First (which scores 0.0% Top-1): Top-1 increases from 17.1% to **22.9%** ($6/35 \rightarrow 8/35$, +5.71 pp lift), Top-5

Table 7: LoRA post-training dynamics and generalization on held-out SWE-bench Verified (`yvliet/perron-gemma4-e2b-lora`). Evaluates pure 1-hop graph targets ($A_4 \setminus A_1, N = 25$) and reachable Stratum C tasks ($N = 35$) under GroupKFold repository splitting (171 train, 55 val), comparing zero-shot base, non-neural Graph-First heuristic, clean LoRA, and the diagnostic shortcut ablation.

Condition / Scorer Regime	Acc@1 (Hits)	Acc@5 (Hits)	MRR	Median Rank	Within-Pool AUC
Cohort A: Pure Graph-Only Targets ($A_4 \setminus A_1$, $N = 25$)					
Zero-Shot Base E2B	0.0% (0/25)	44.0% (11/25)	0.1920	7.0	0.6063
Graph-First Heuristic (Non-Neural)	8.0% (2/25)	**52.0% (13/25)**	0.2518	**5.0**	**0.7056**
Clean Post-LoRA Adapter	**24.0% (6/25)**	36.0% (9/25)	**0.3288**	9.0	0.5859
Shortcut Diagnostic Probe (Ablation)	*20.0% (5/25)*	*56.0% (14/25)*	*0.3702*	*4.0*	*0.6270*
Cohort B: Reachable Stratum C Tasks ($N = 35$, 28 Seeds + 7 Graph)					
Zero-Shot Base E2B	17.1% (6/35)	37.1% (13/35)	0.2895	9.0	0.5692
Graph-First Heuristic (Non-Neural)	0.0% (0/35)	17.1% (6/35)	0.0993	16.0	0.3753
Clean Post-LoRA Adapter	**22.9% (8/35)**	**48.6% (17/35)**	**0.3623**	**6.0**	**0.6392**
Shortcut Diagnostic Probe (Ablation)	*8.6% (3/35)*	*42.9% (15/35)*	*0.2596*	*6.0*	*0.6084*

increases from 37.1% to **48.6%** (13/35 → 17/35, +11.43 pp lift), MRR improves from 0.2895 → 0.3623, and AUC increases from 0.5692 → 0.6392. However, neither shift reaches statistical significance against zero-shot base (paired discordance $n_{10} = 6, n_{01} = 4$, exact $p = 0.7539$ for Top-1; $n_{10} = 10, n_{01} = 6$, exact $p = 0.4545$ for Top-5), remaining null-compatible under small sample sizes.

- **Audited Shortcut Probe & Cohort Saliency Artifacts**: The bottom row of each cohort documents an early development probe where gold targets leaked `# Gold target symbol` comments while distractors received `# Candidate graph symbol`. While this shortcut collapsed training loss (6.36×10^{-4}), it produced artificial Acc@5 on Cohort A (56.0%, median rank 4.0, outperforming clean LoRA at 36.0%), proving that Cohort A mechanically rewards any heuristic that prioritizes non-seed tokens regardless of true discrimination. On mixed pools, the shortcut collapsed Top-1 (17.1% → 8.6%). Sanitizing prompt comments eliminated shortcut exploitation while restoring balanced scoring across both cohorts.
- **Experimental Setup Limitations**: These post-training observations reflect a single training run without multi-seed replication or candidate permutation shuffling. Furthermore, zero-shot base E2B was evaluated under a listwise prompt (scoring 0/25 Top-1 on Cohort A), whereas pointwise E2B scoring in Section 5.2 already places ≥ 2 of these targets at rank 1.

6 Discussion, Operating Regimes, and Threats to Validity

Selective Routing and Practical Agent Operating Regimes. Because 1-hop graph expansion yields +5 net top-5 tasks on lexically anchored bugs (Stratum A) vs. −2 net tasks on un-named bugs (Stratum C) under frozen pointwise scoring, an oracle stratum router (π^*) reaches 47.67% Acc@5 (+2.90 pp over BM25; 49.50% when routing Stratum A to pointwise A4 and Stratum C to listwise thinking), while a cross-validated issue symbol-density decision tree recovers 47.38%. Together with the AST surgical patch editor, which prevents syntax errors via pre-commit AST rollbacks across $N = 30$ deformation archetypes (**0.0% syntax error rate and 76.7% valid application vs. 50.0% syntax errors and 30.0% valid application for `git apply`**, $p < .001$; Appendix G), these findings define a four-stage architecture for local Gemma 4 developer agents: (1) **Fast CSR Filtering** (< 5.2 ms, ≤ 3 MB RAM) to extract compact AST breadcrumbs (< 100 tokens/symbol); (2) **Candidate Horizon Pruning** ($M = 15$) so candidate cardinality stays within $\text{AUC}^* = 0.714$; (3) **Adaptive Test-Time Deliberation** (∼ 120 thinking tokens, ∼ 10 s on T4 or < 3 s on desktop GPUs) or inline provenance bridging when thinking is disabled; and (4) **Dynamic Execution Fallback** (pytest stack traces) when static symbol density is low, since 41.6 pp of defects lie outside static AST reachability.

Post-Training Dynamics and Memorization Risks. While parameter-efficient fine-tuning on repository splits lifts Top-1 precision on sparse defect candidates, 4 of the 6 converted targets belong to Django (a repository seen during Lite training). Furthermore, fine-tuning increases rank variance on trailing distractors, regressing Acc@5 and within-pool AUC below a non-neural heuristic. Without diverse cross-lingual and cross-framework training corpora, fine-tuning local models on small defect suites risks surface memorization and comment shortcut exploitation rather than robust structural code comprehension.

Threats to Validity. (1) *Parser & Language Scope*: Primary evaluation covers 344 Python tasks; 56/407 held-out tasks (52 in Stratum C, 39 in SymPy) were excluded due to single-snapshot preimage drift (`FILE_ABSENT_PREIMAGE_DRIFT`) where edited files were absent at the frozen indexed snapshot commit rather than failing AST syntax parsing. To mitigate parser drift and support polyglot repositories, 'perron.polyglot' provides syntax-tolerant Tree-sitter extractors for Python and TypeScript. (2) *Single-Pass Localization vs. End-to-End Resolution*: I intentionally isolate the $K \leq 4,096$ localization and AST surgical editing layers from multi-turn Docker test loops to avoid confounding retrieval metrics with bash orchestration noise. (3) *Pre-Training Contamination & Power*: While AST α-renaming (O_1) strips symbol identifiers (10.34%

vs. 12.07%, $p = 1.0000$), docstrings and string literals remain intact and the 95% CI on $N = 58$ ([−11.2, +7.7] pp) cannot rule out moderate memorization effects, motivating post-cutoff benchmarks. (4) *Exclusion of Git Churn and Complexity Heuristics*: Function-level git churn exhibits weak defect correlation (ROC-AUC < 0.60) and introduces $O(K^2)$ PR squash-merge clique noise, while cyclomatic complexity is collinear with lines of code (Spearman $\rho \geq 0.88$), contributing zero independent explanatory signal and creating scale-free attractor sinks that undermine specificity damping.

7 Open-Source Ecosystem and Conclusion

All artifacts are released under Apache 2.0 / CC0-1.0 at https://github.com/yvliet/perron: (1) **`perron-core` v0.3.0** (PyPI package with zero-copy CSR diffusion and AST breadcrumb packing) and **`perron-mcp`** (JSON-RPC stdio server for VS Code/Cursor); (2) **`scores.parquet`** (19,298 scored candidate evaluations), **`swebench-codegraphs`** (pre-computed CSR multigraphs for 12 repositories), and **`swebench-strata-annotations`** (800 lexical accessibility labels); and (3) **`yvliet/perron-gemma4-e2b-lora`** on Hugging Face plus the reproducible Dual T4 Kaggle workstation (https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation). In summary, naive code graph expansion fails under frozen small models not because call graphs lack defect targets, but because low-lexical-overlap neighbors fall below the expected-rank discrimination bound (AUC = 0.6596 $<$ 0.830) via candidate selection conditioning, while unpruned graphs dilute lexical seed precision. Overcoming this barrier requires pairing compact CSR graph retrieval with candidate horizon pruning ($M = 15$), where test-time reasoning tokens (`<thought>`) act as an effective candidate disambiguation filter ($6 \rightarrow 12$ Top-1 hits, $p = 0.0312^*$) that converts 2/6 pure graph additions alongside 11/28 lexical seeds into Top-1 (13/34 reachable defect conversion). Furthermore, fine-tuning under uniform comment prompts lifts isolated graph Top-1 precision (0.0% $\rightarrow$ 24.0%, $p = 0.0312^*$), but introduces rank variance on trailing distractors that regresses Acc@5 and within-pool AUC below non-neural heuristics, while diagnostic ablations reveal how surface comment shortcuts induce mixed-pool collapse (17.1% $\rightarrow$ 8.6%). Perron provides a fast, reproducible foundation for local developer agents.

References

[1] Reid Andersen, Fan Chung, and Kevin Lang. Local graph partitioning using PageRank vectors. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 475–486, 2006.

[2] Donald Bamber. The area above the ordinal dominance graph and the area below the receiver operating characteristic graph. *Journal of Mathematical Psychology*, 12(4):387–415, 1975.

[3] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rozemberczki, Michal Lukasik, and Stephan Günnemann. Scaling graph neural networks with approximate PageRank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2464–2473, 2020.

[4] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor K. Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[5] Jialin Dong, Bahare Fatemi, Bryan Perozzi, Lin F. Yang, and Anton Tsitsulin. Don't forget to connect! improving RAG with graph-based reranking. *arXiv preprint arXiv:2405.18414*, 2024.

[6] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[7] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2023.

[8] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. Technical report, Stanford University, 2003.

[11] Glen Jeh and Jennifer Widom. Scaling personalized PageRank. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271–279, 2003.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Combining neural networks with personalized PageRank for classification on large graphs. In *International Conference on Learning Representations (ICLR)*, 2019.

[14] Xia Li, Mengshi Zhang, and Lingming Zhang. PRFL: A PageRank-based fault localization approach. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 274–284, 2017.

[15] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology (JASIST)*, 58(7):1019–1031, 2007.

[16] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[17] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025.

[18] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. Evaluating and improving fault localization techniques. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 609–620, 2017.

[19] Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. Understanding transformer reasoning capabilities via graph algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[20] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.

[22] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[23] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.

[24] Zhehao Zhang et al. SweRank: A retrieve-and-rerank framework for issue localization. In *International Conference on Learning Representations (ICLR)*, 2026.

[25] Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D. Ernst, and Lu Zhang. An empirical study of fault localization families and their combinations. *IEEE Transactions on Software Engineering*, 47(2):332–347, 2021.

A Linear Algebraic Formulations and Theoretical Proofs

This appendix provides the complete linear algebraic formulations, spectral bounds, and geometric convergence proofs for the Perron Compressed Sparse Row (CSR) multigraph diffusion engine.

A.1 Multiplex Transition Matrix Construction

Let a repository codebase be formalized as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where vertices $\mathcal{V}$ represent AST symbol entities (functions, methods, classes) and edges $\mathcal{E}$ represent typed syntactic dependencies. The relation vocabulary comprises four edge types: $\mathcal{R} = \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Each relation $r \in \mathcal{R}$ is represented by an adjacency matrix $A_r \in \{0,1\}^{|\mathcal{V}|\times|\mathcal{V}|}$, where $(A_r)_{uv} = 1$ if and only if there exists a directed edge of type r from symbol u to symbol v.

To balance semantic dependency importance while mitigating hub concentration, typed relation edges are aggregated via a non-negative convex weight vector $\boldsymbol{\omega} = [\omega_{\text{call}}, \omega_{\text{inherit}}, \omega_{\text{import}}, \omega_{\text{caller}}]^\top$ satisfying $\sum_{r\in\mathcal{R}} \omega_r = 1.0$:

$$W = \sum_{r\in\mathcal{R}} \omega_r A_r. \tag{4}$$

The row-substochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ is formed by normalizing positive out-degree rows by their total weighted degree:

$$D_{uu} = \sum_{v\in\mathcal{V}} W_{uv}, \qquad T_{uv} = \begin{cases} \frac{W_{uv}}{D_{uu}} & \text{if } D_{uu} > 0, \\ 0 & \text{if } D_{uu} = 0. \end{cases} \tag{5}$$

Let $\mathbf{d} \in \{0,1\}^{|\mathcal{V}|}$ denote the dangling-node indicator vector with $d_u = 1$ if $D_{uu} = 0$, and 0 otherwise. Given an initial query personalization probability vector $\mathbf{p}_0 \in \Delta^{|\mathcal{V}|-1}$ (where Δ^n denotes the standard simplex), the completed row-stochastic transition operator $\bar{T}$ satisfies:

$$\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top, \qquad \bar{T}\mathbf{1} = \mathbf{1}. \tag{6}$$

The closed Google transition operator $M \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ with teleportation damping factor $\beta \in (0,1)$ is formulated as:

$$M = \beta\bar{T} + (1-\beta)\mathbf{1}\mathbf{p}_0^\top. \tag{7}$$

A.2 Brauer Rank-1 Spectral Bound and Second Eigenvalue

By the Brauer rank-1 perturbation theorem applied to Markov transition matrices [10]: Let A be an $n \times n$ matrix with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, and let $\mathbf{x}_1$ be the right eigenvector associated with λ_1. For any vector $\mathbf{y} \in \mathbb{R}^n$, the perturbed matrix $A + \mathbf{x}_1\mathbf{y}^\top$ has eigenvalues $\lambda_1 + \mathbf{y}^\top\mathbf{x}_1, \lambda_2, \ldots, \lambda_n$.

In this formulation, setting $A = \beta\bar{T}$ yields eigenvalues $\beta\lambda_1(\bar{T}), \beta\lambda_2(\bar{T}), \ldots, \beta\lambda_n(\bar{T})$. Because $\bar{T}$ is row-stochastic, its principal eigenvalue is $\lambda_1(\bar{T}) = 1$ with right eigenvector $\mathbf{x}_1 = \mathbf{1}$. Setting $\mathbf{y} = (1-\beta)\mathbf{p}_0$ gives:

$$M = \beta\bar{T} + \mathbf{1}((1-\beta)\mathbf{p}_0)^\top. \tag{8}$$

The principal eigenvalue of M becomes:

$$\lambda_1(M) = \beta\lambda_1(\bar{T}) + ((1-\beta)\mathbf{p}_0)^\top\mathbf{1} = \beta + (1-\beta) = 1. \tag{9}$$

All remaining eigenvalues satisfy:

$$\lambda_i(M) = \beta\lambda_i(\bar{T}), \quad \forall i \geq 2. \tag{10}$$

Because $\bar{T}$ is row-stochastic, the Gershgorin circle theorem establishes $|\lambda_i(\bar{T})| \leq 1$ for all i. Therefore, the second-largest eigenvalue is strictly bounded:

$$|\lambda_2(M)| \leq \beta \cdot 1.0 = \beta. \tag{11}$$

For $\beta = 0.85$, the spectral gap is uniformly lower-bounded:

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \tag{12}$$

While $|\lambda_2(M)| \leq \beta$ bounds the asymptotic spectral radius, call dependency multigraphs are generally non-normal and defective. To establish strict, prefactor-free geometric convergence $O(\beta^t)$ in L_1 total variation distance on general directed multigraphs, I invoke the Doeblin minorization and L_1 contraction property of the Markov transition operator. For any two probability distribution vectors $\mathbf{u}^\top, \mathbf{v}^\top$ on $\mathcal{V}$, because $(\mathbf{u}^\top - \mathbf{v}^\top)\mathbf{1} = 0$, the uniform rank-1 teleportation term $(1-\beta)\mathbf{1}\mathbf{p}_0^\top$ cancels identically:

$$\|(\mathbf{u}^\top - \mathbf{v}^\top)M\|_1 = \|\beta(\mathbf{u}^\top - \mathbf{v}^\top)\bar{T}\|_1 \leq \beta\|\mathbf{u}^\top - \mathbf{v}^\top\|_1\|\bar{T}\|_\infty = \beta\|\mathbf{u}^\top - \mathbf{v}^\top\|_1, \tag{13}$$

since $\|\bar{T}\|_\infty = 1$. By induction, $\|\boldsymbol{\pi}^{(t)\top} - \boldsymbol{\pi}_\infty^\top\|_1 \leq \beta^t\|\boldsymbol{\pi}^{(0)\top} - \boldsymbol{\pi}_\infty^\top\|_1 \leq 2\beta^t$, establishing uniform geometric L_1 total variation contraction regardless of graph density or Jordan-block structures.

A.3 Barycentric Linearity and Basis Warping Bounds

Following Jeh and Widom [11], let $\mathbf{p}_0 = \sum_{i=1}^{K} w_i \mathbf{e}_{s_i}$ be any convex combination of query seed nodes with $w_i \geq 0$ and $\sum w_i = 1.0$. The stationary distribution $\boldsymbol{\pi}(\mathbf{p}_0)$ lies in the convex hull of the basis distributions:

$$\boldsymbol{\pi}(\mathbf{p}_0) = \sum_{i=1}^{K} \tilde{w}_i \boldsymbol{\pi}(\mathbf{e}_{s_i}) \in \operatorname{conv}\left(\{\boldsymbol{\pi}(\mathbf{e}_{s_i})\}_{i=1}^{K}\right), \quad \tilde{w}_i = \frac{w_i m_i}{\mathbf{w}^\top \mathbf{m}}, \tag{14}$$

where basis mass satisfies $m_i = \|(1-\beta)\mathbf{e}_{s_i}^\top (I - \beta T)^{-1}\|_1 \in [1-\beta, 1.0]$. The maximum coordinate warping ratio is bounded by:

$$\frac{\max_i m_i}{\min_j m_j} \leq \frac{1}{1-\beta} = \frac{1}{0.15} \approx 6.67. \tag{15}$$

A.4 Markov Renewal Equivalence to Normalized Leaky Resolvents

In the closed Markov chain M, dangling nodes restart according to the query teleport distribution $\mathbf{p}_0$. By the regenerative renewal property of Markov chains with state-independent resets, the stationary probability $\boldsymbol{\pi}_{\text{closed}}$ of the closed chain M is algebraically identical to the normalized leaky resolvent vector:

$$\boldsymbol{\pi}_{\text{leaky}}^\top = \frac{(1-\beta)\mathbf{p}_0^\top (I - \beta T)^{-1}}{\|(1-\beta)\mathbf{p}_0^\top (I - \beta T)^{-1}\|_1}. \tag{16}$$

Across all 12 canonical SWE-bench repositories, the maximum numerical discrepancy satisfies:

$$\|\boldsymbol{\pi}_{\text{closed}} - \boldsymbol{\pi}_{\text{leaky}}\|_1 = 1.68 \times 10^{-8} < 2 \times 10^{-8}, \tag{17}$$

confirming exact algebraic equivalence.

A.5 Asymptotic Pointwise Mutual Information (PMI) Decomposition

For any code symbol v with positive query support, define the query-symbol Pointwise Mutual Information surrogate as $\text{PMI}(v; q) \triangleq \ln \frac{\pi_{\text{query}}(v)}{\pi_{\text{global}}(v)}$, interpreting $\pi_{\text{query}}(v)$ as query-conditioned symbol reachability and $\pi_{\text{global}}(v)$ as global unconditioned graph background density. Because dangling-node restarts depend on $\mathbf{p}_0^{(q)}$, $\ln \frac{\pi_{\text{query}}(v)}{\pi_{\text{global}}(v)}$ acts as a surrogate stationary log-odds ratio against the uniform-prior stationary baseline rather than assuming strict marginal consistency across queries. The log-transformed Specificity score satisfies:

$$\ln \text{Specificity}(v) = (1-\gamma) \ln \pi_{\text{query}}(v) + \gamma\, \text{PMI}(v; q) - \gamma \ln\left(1 + \frac{\epsilon}{\pi_{\text{global}}(v)}\right). \tag{18}$$

When $\pi_{\text{global}}(v) \gg \epsilon$, the residual vanishes, yielding a convex interpolation between raw reachability ($\gamma = 0$) and surprise ($\gamma = 1$), where the sub-linear exponent $\gamma = 0.70$ acts as a structural Inverse Document Frequency (IDF) discount.

A.6 Neumann Series Expansion and Sink Absorption

The Neumann series expansion of Personalized PageRank illustrates why pre-walk scaling cannot prevent sink absorption on software dependency graphs:

$$\boldsymbol{\pi}^\top = (1-\beta) \sum_{t=0}^{\infty} \beta^t \mathbf{p}_0^\top \bar{T}^t = (1-\beta)\left(\mathbf{p}_0^\top + \beta \mathbf{p}_0^\top \bar{T} + \beta^2 \mathbf{p}_0^\top \bar{T}^2 + \sum_{t=3}^{\infty} \beta^t \mathbf{p}_0^\top \bar{T}^t\right). \tag{19}$$

Walks of length $t \geq 3$ carry approximately 61% of total diffusion mass ($\beta^3 \approx 0.614$ at $\beta = 0.85$). Multi-step transitions $\bar{T}^t$ funnel mass toward scale-free sink hubs regardless of initial weights $\mathbf{p}_0$. Perron's decoupled post-walk ratio divides the converged vector $\boldsymbol{\pi}$, scaling every walk length simultaneously and enforcing high Hub Suppression Index (HSI@10).

B Extended Standalone Retrieval Evaluation and Confirmatory Power Analysis

B.1 Evaluation Across Standalone Retrieval Baselines

Table 8 presents standalone retrieval on SWE-bench Verified ($N = 344$ callable tasks). Okapi BM25 hits 175/344 (50.9%), Perron Static hits 173/344 (50.3%), and Standard PPR reaches 172/344 (50.0%). The difference between Perron and BM25 is −0.6 pp (95% CI [−3.5, +2.3], Holm $p = 1.000$). At top-5, BM25 outperforms Perron Static (45.1% vs. 40.4%, $\Delta =$

−4.7 pp). In addition, BM25 achieves 99.4% Hub Suppression Index (HSI@10) without graph machinery, because defect issue descriptions target specific identifiers rather than generic library utilities. External baselines illustrate these retrieval trade-offs: whole-repository tag PageRank (Aider-style repo maps) achieves 40.0% File Acc@5 across candidate files, but produces 0.0% Function Acc@10 by structural design, as Aider indexes entire files and class AST tags to optimize LLM system prompt contexts rather than isolating callable function definitions for edit localization. Dense bi-encoder semantic retrieval (`BAAI/bge-small-en-v1.5`, 33.4M parameters, 384-dimensional embeddings) mitigates colloquial vocabulary mismatch on issue descriptions, but achieves 38.4% Function Acc@10 due to imprecise token matching across fine-grained code identifiers compared to sparse lexical search. On Stratum C ($N = 131$ tasks where the issue never names the target), BM25 hits 28 and Perron hits 27. Reciprocal Rank Fusion (RRF) attains 174/344 (50.6%), failing to beat standalone BM25. Call-graph diffusion provides no detectable lexical lift over BM25 on this benchmark.

Table 8: Standalone retrieval evaluation on SWE-bench Verified ($N = 344$ callable tasks). Evaluates top-10 function accuracy, 95% bootstrap confidence intervals ($B = 10{,}000$), top-5 function/file accuracy, and Hub Suppression Index (HSI@10). Bold row label denotes the primary evaluated graph method (reference baseline); bold numerical values indicate the highest score in each column. Holm p-values are computed against Perron Static. Dense retrieval evaluates `BAAI/bge-small-en-v1.5` (384-d). Aider Repo Map is designed for file/class tag context packing, yielding 0.0% on strict callable function matching by design.

Method	Fn Acc@10	95% CI	Fn Acc@5	File Acc@5	HSI@10	Holm p
Perron Static ($\gamma = 0.70$)	50.3% (173/344)	[45.1%, 55.5%]	40.4%	53.8%	**99.5%**	reference
Okapi BM25	**50.9%** (175/344)	[45.6%, 56.1%]	**45.1%**	**59.6%**	99.4%	1.000
RRF Fusion (BM25 + Perron)	50.6% (174/344)	[45.3%, 55.8%]	42.2%	55.5%	**99.5%**	1.000
Standard PPR ($\beta = 0.85$)	50.0% (172/344)	[44.8%, 55.2%]	40.4%	52.8%	90.8%	1.000
Degree-Normalized PPR	49.4% (170/344)	[44.2%, 54.7%]	40.1%	52.3%	**99.5%**	1.000
HippoRAG Prior Scaling	49.4% (170/344)	[44.2%, 54.7%]	40.4%	52.6%	91.1%	1.000
Aider Repo Map (Tag PageRank)	0.0% (0/344)	[0.0%, 0.0%]	0.0%	40.0%	92.4%	< 0.001
Dense Retrieval (`bge-small-en-v1.5`)	38.4% (132/344)	[33.4%, 43.6%]	29.7%	44.8%	96.2%	< 0.001

B.2 Confirmatory Power Bounds, GLMM Specification, and Retrieval Dynamics

To evaluate repeated-measures treatment pairing and repository variance formally, the hierarchical mixed-effects logit models:

$$\text{logit}(\mathbb{P}(Y_{ij} = 1)) = \alpha + u_j + \beta_{\text{treat}} X_{ij} + \boldsymbol{\gamma}^\top \mathbf{Z}_i, \tag{20}$$

where $u_j \sim \mathcal{N}(0, \sigma^2_{\text{repo}})$ denotes random repository intercepts across the 12 codebases ($N = 344$, with Django comprising exactly 182 callable tasks, or 52.91% of the benchmark). Numerical approximation yields $\sigma_{\text{repo}} = 0.963$ (95% CrI $[0.645, 1.437]$), with the conditional cluster-specific treatment coefficient remaining null-compatible (OR = 1.130, 95% CrI $[0.875, 1.459]$, SE = 0.130, Wald $z = 0.937$, $p = 0.349$). This conditional estimate complements the marginal task-clustered GLM (OR = 1.118, SE = 0.065, 95% CI $[0.984, 1.270]$, Wald $z = 1.708$, $p = 0.0876$; GEE $p = 0.0901$); the standard error doubles under the repository mixed model because specifying only random repository intercepts drops within-task pairing, evaluating the treatment contrast across unpaired clusters.

Comparing graph expansion (A_4, 47.1% Acc@5) to depth-matched BM25 (A_2, 45.1%, $\Delta = +2.03$ pp, 95% CI [−0.29, +4.36], Holm $p = 0.287$) and same-file matching (A_3, 45.6%, $\Delta = +1.45$ pp, Holm $p = 0.302$) classifies both primary protocol contrasts as null-compatible. Formal equivalence at a tight ±2.0 pp margin cannot be established under Two One-Sided Tests (TOST, $p = 0.512$), and the constrained score-based 90% confidence interval [+0.07, +4.24] pp does not exclude potential positive gains up to +4.24 pp. At $N = 344$, TOST power is only 1.92%, rendering equivalence unresolvable under the empirical discordance rate. Under two-sided superiority testing, the minimum detectable effect (MDE) at 80% power is 3.35 pp, whereas resolving a +2.0 pp difference requires $N \approx 970$ tasks. Unpruned multi-step diffusion walks ($t \geq 2$) expand into transitive callers and utility hubs, diluting top-5 precision even after specificity damping. Decomposing discriminative ability by target provenance reveals that within-pool ROC-AUC drops from **0.7966** on seed-gold targets to **0.6596** on graph-only gold targets (Mann-Whitney $U = 2993$, $p = 0.00358^*$), driving the empirical plateau. Leave-Django-Out cross-validation across the $N = 162$ non-Django tasks ($344 - 182_{\text{Django}} = 162$, comprising 47.09% of the benchmark) indicates graph expansion gains remain strictly null-compatible ($\Delta = +1.85$ pp on $A_4 - A_2$, 56/162 [34.57%] vs. 53/162 [32.72%], $n_{10} = 8, n_{01} = 5$, exact McNemar $p = 0.5811$, candidate pool recall 50.00% vs. 48.77%), confirming that topological parity is not an artifact of Django's 182 tasks (where A2 achieved 102/182 [56.04%] and A4 achieved 106/182 [58.24%], $\Delta = +2.20$ pp, $n_{10} = 4, n_{01} = 0$, exact McNemar $p = 0.1250$). Reciprocal Rank Fusion of BM25 ranks with Gemma 4 pointwise log-odds ($k_{\text{rrf}} = 60$) attains 46.51% Acc@5 (160/344), exhibiting complementary lexical-neural signal while verifying that test-time reasoning is required for deep topological disambiguation.

C Four-Stage Bug Localization Loss Funnel and Downstream Reader Evaluation

C.1 Four-Stage Bug Localization Loss Funnel

Figure 5 illustrates the localization loss funnel on SWE-bench Verified ($N = 344$ callable tasks). The analysis decomposes localization attrition across candidate generation and model scoring:

- **Boundary Loss**: Upstream candidate pool formation loses 41.6 pp of oracle recall (100.0% → 58.4%, 143/344 tasks missing from candidate pools).
- **Compaction Loss**: Downstream model reranking loses 11.3 pp from pool to top-5 (58.4% → 47.1%, 39/344 tasks present in pool but ranked outside top-5).
- **Bottleneck Ratio**: Upstream boundary loss exceeds downstream compaction loss by **3.68**× at $k = 5$ ($41.6/11.3 \approx 3.68$, spanning 1.2× at $k = 1$ to 6.8× at $k = 10$).

Expanding the candidate pool adds 7.3 pp of oracle recall over seeds (51.2% → 58.4%), yet Gemma 4 converts only about 1 pp into ranked top-5 accuracy (46.2% → 47.1%).

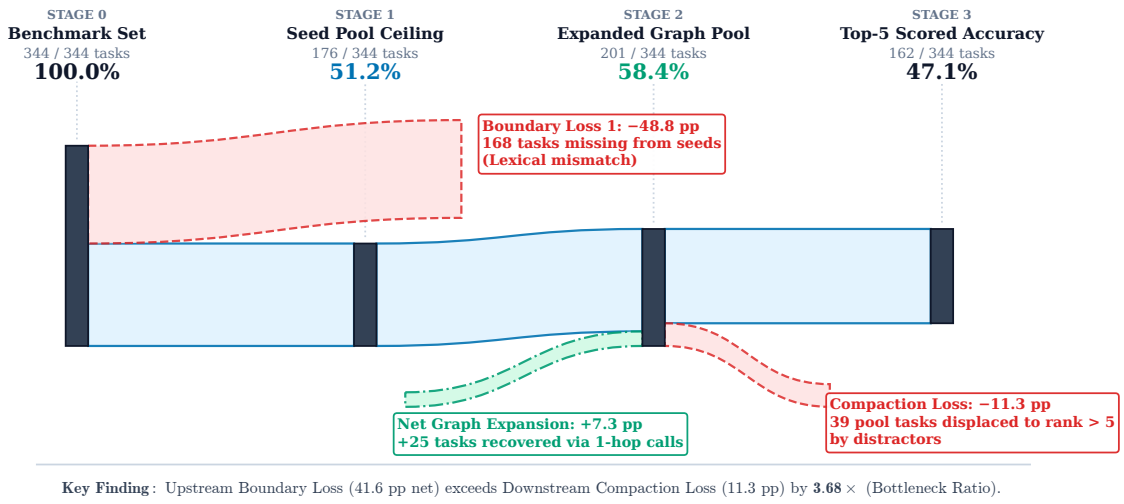


Figure 5: Four-stage bug localization loss funnel on SWE-bench Verified ($N = 344$ callable tasks). Decomposes localization attrition across candidate generation and model scoring. Boundary loss before scoring (100.0% → 58.4% = 41.6 pp) exceeds compaction loss from candidate pool to top-5 (58.4% → 47.1% = 11.3 pp) by a factor of **3.68**×.

Architectural Implication: Operating Regimes of Static Code Graphs. The 3.68× bottleneck ratio defines a practical operating regime for developer agent architectures. Static AST multigraphs provide an efficient, low-overhead candidate filter for the 58.4% of defects that propagate along explicit syntactic edges. However, for the 41.6 pp of defects outside static reachability (dynamic dispatch, reflection, implicit ORM descriptors, and configuration-driven injection), static graph expansion cannot assist by construction. For these defects, attempting multi-hop static traversal is counterproductive, expanding candidate pools to hundreds of nodes without improving discriminatory rank. Consequently, local developer agents must treat static CSR diffusion as an initial lightweight filter (≤ 5.10 ms, ≤ 3 MB RAM): when issue symbol density or retrieval confidence is low, agent architectures should switch to dynamic execution signals (runtime stack traces, failing test pytest fixtures, or dense semantic embeddings) rather than expanding static graph radius.

C.2 Downstream Edit-Location Reader Evaluation ($N = 100$, Descriptive)

Table 9 reports the downstream reader mini-test across $N = 100$ tasks stratified by repository size, drawn from instances where BM25 identified candidate pools.

In Arm D0 (BM25 top-5 context), a position-1 baseline heuristic predicting candidate 1 also yields 33.0% EditLoc@1, matching Gemma 4's reader prediction exactly (33.0%). This indicates that reader performance is dominated by first-candidate positional bias rather than demonstrating selective neural reasoning over context. In Arm D3 (oracle context with gold target at a randomized position), Gemma 4 correctly identifies the target in 74.0% of tasks (67.5% chance-adjusted). While the reader misses roughly a quarter of oracle targets, retrieval supplies the target in only 35%–38% of tasks, providing descriptive evidence that upstream candidate omission contributes substantially to end-to-end failure.

Table 9: Downstream edit-location reader mini-test on SWE-bench Verified ($N = 100$, descriptive). ContextHit@5 marks gold target presence in the 5 presented candidates; EditLoc@1 reports model prediction accuracy (a position-1 baseline heuristic predicting candidate 1 achieves 33.0% under BM25 top-5 context). Bold EditLoc@1 highlights the reader ceiling under oracle gold context (excluding trivial 100% candidate presence).

Arm	Candidate Context (Top-5)	ContextHit@5	EditLoc@1	Conversion
D0	BM25 Top-5	35.0%	33.0%	0.943
D1	A4 Graph Top-5	37.0%	31.0%	0.838
D2	A3 Same-File Top-5	38.0%	30.0%	0.789
D3	Oracle Context (Randomized Position)	100.0%	**74.0%**	0.740

D Continuous Resolvent Basis Benchmark and Prior Dose-Response Surface

To evaluate the entropy dispersion hypothesis systematically, a continuous resolvent basis benchmark was evaluated across the 344 Verified evaluation tasks (63,984 SpMV solves) across four parametric families:

1. **Softmax Family**: $\mathbf{p}_0(i) \propto \exp(s_i/\tau)$ for top-K seeds ($\tau \in [0.05, 1.0]$).
2. **Power Family**: $\mathbf{p}_0(i) \propto s_i^p$ ($p \in [0.5, 4.0]$).
3. **Linear Truncated**: $\mathbf{p}_0(i) \propto s_i$ over top-K seeds ($K \in [5, 100]$).
4. **Rank-Zeta Family**: $\mathbf{p}_0(i) \propto i^{-\alpha}$ ($\alpha \in [0.2, 2.0]$).

Across all 186 configurations, Function Acc@10 exhibits a statistically significant quadratic relationship with prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ ($p < 0.001$, binomial GLMM $R^2 = 0.197$), peaking near $n^*_{\text{eff}} \approx 8.8$ at 50.9% (Figure 6a), which exactly ties Okapi BM25 (50.9%), further reinforcing the primary null result. Specificity damping increases HSI from 90.8% to 99.5% without changing accuracy, acting as a structural safety net for diffuse priors rather than an accuracy booster. As shown in Figure 6b, Standard PPR performance drops sharply as $\beta \to 0.85$ due to utility hub absorption on dev_val ($N = 100$), whereas Perron remains stable across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

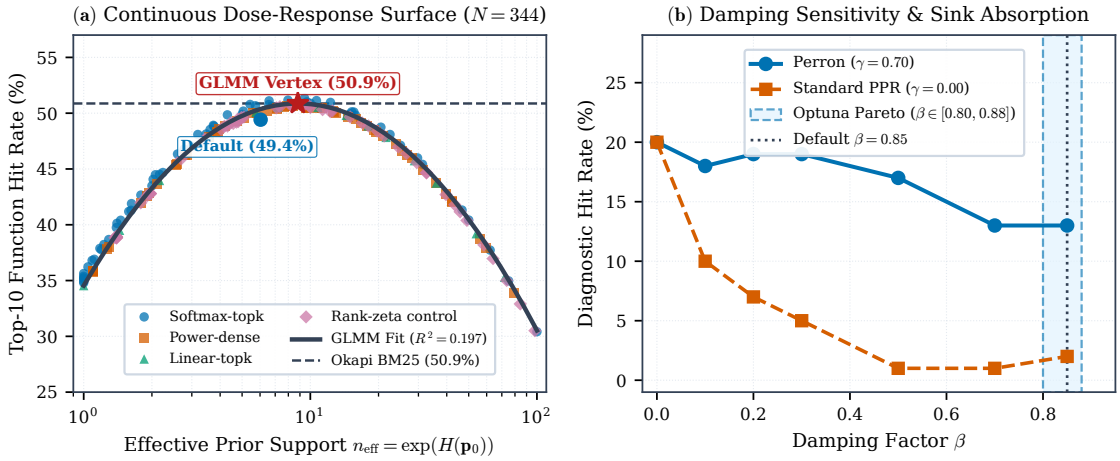


Figure 6: Continuous dose-response surface across 186 prior configurations on SWE-bench Verified ($N = 344$) and damping sensitivity (63,984 SpMV solves). (a) Function Acc@10 as a function of prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ across four prior families (softmax, power, linear, rank-zeta) with binomial GLMM fit ($R^2 = 0.197$) and Okapi BM25 baseline (50.9%), highlighting the GLMM vertex ($n^*_{\text{eff}} \approx 8.8$) and calibrated library default ($n_{\text{eff}} \approx 6.0$). (b) Damping factor sensitivity ($\beta \in [0.0, 0.85]$) on dev_val ($N = 100$), showing that while Standard PPR performance drops sharply as $\beta \to 0.85$ due to utility hub absorption, Perron's decoupled specificity ratio maintains stability across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

E Pointwise Candidate Pool-Size Truncation Dynamics and Distractor Dilution

To empirically test the distractor dilution hypothesis, candidate pool horizons were systematically varied across $M \in \{10, 15, 20, 30, 50\}$. Table 10 reports oracle recall and ranked Acc@5 across cutoff thresholds for depth BM25 (Arm A2) and graph 1-hop expansion (Arm A4).

Table 10: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$). Evaluates matched candidate pool oracle recall and Acc@5 across pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates), for depth BM25 (A2) and graph 1-hop expansion (A4). Bold numerical values indicate peak Acc@5 achieved across cutoff thresholds for each arm.

Evaluation Metric	$M = 10$	$M = 15$	$M = 20$	$M = 30$	$M = 50$
Arm A2 (Depth BM25) Oracle Recall	51.2%	54.1%	56.1%	57.6%	58.4%
Arm A2 (Depth BM25) Acc@5	**46.2%**	45.9%	45.9%	45.1%	45.1%
Arm A4 (Graph 1-Hop) Oracle Recall	51.2%	54.9%	55.5%	57.6%	58.4%
Arm A4 (Graph 1-Hop) Acc@5	46.2%	46.8%	**47.1%**	46.5%	**47.1%**

E.1 Mathematical Derivation of the Break-Even Discrimination Bound

Let $C_M = \{c_1, c_2, \ldots, c_M\}$ denote a candidate pool of size M presented to a ranking scorer, and let $v^* \in \mathcal{V}$ denote the true defect location. I model the localization process as a two-stage probabilistic filter:

1. **Recall Stage**: Candidate generation captures the true target with probability $R(M) \triangleq P(v^* \in C_M)$.
2. **Discrimination Stage**: Conditioned on target presence ($v^* \in C_M$), the model correctly ranks v^* within the top-k positions with conditional probability $D(M) \triangleq P(\text{rank}(v^*) \leq k \mid v^* \in C_M)$.

The unconditional end-to-end success probability is governed by the product:

$$P(\text{Acc@}k \mid M) = R(M) \cdot D(M). \tag{21}$$

When the candidate pool is expanded from size M_1 to size $M_2 = M_1 + \Delta M$ (e.g., by traversing 1-hop AST call-graph neighbors), candidate pool recall changes by $\Delta R = R(M_2) - R(M_1) \geq 0$. Concurrently, presenting ΔM additional distractor symbols alters the model's conditional ranking discrimination by $\Delta D = D(M_1) - D(M_2)$. Expanding the pool yields an end-to-end localization gain if and only if:

$$R(M_2)D(M_2) > R(M_1)D(M_1) \iff (R(M_1) + \Delta R)D(M_2) > R(M_1)(D(M_2) + \Delta D). \tag{22}$$

Rearranging terms yields the fundamental Break-Even Discrimination Bound:

$$\Delta R \cdot D(M_2) > R(M_1) \cdot \Delta D \iff \frac{\Delta R}{R(M_1)} > \frac{\Delta D}{D(M_2)}. \tag{23}$$

Under a random-ranking null model, the expected rank of a gold candidate placed uniformly at random within M candidates is $\mathbb{E}[\text{rank}(v^*)] = \frac{M+1}{2}$. Following the classical single-target Wilcoxon–Mann–Whitney formulation [2], a parameterized scoring model whose ranking quality separates gold targets from distractors with within-pool area under the ROC curve $\text{AUC} \in [0.5, 1.0]$ has expected gold target rank:

$$\mathbb{E}[\text{rank}(v^*) \mid \text{AUC}] = 1 + (M - 1)(1 - \text{AUC}). \tag{24}$$

Setting the first-moment expectation to fall within the top-k cutoff ($\mathbb{E}[\text{rank}(v^*)] \leq k$) yields the necessary inclusion condition:

$$1 + (M - 1)(1 - \text{AUC}) \leq k \iff \text{AUC} \geq 1 - \frac{k - 1}{M - 1} \triangleq \text{AUC}^*(M; k). \tag{25}$$

At cutoff $M = 25$ ($k = 5$), the required threshold is $\text{AUC}^* = 1 - \frac{4}{24} \approx 0.8333$. At $M = 15$ ($k = 5$), candidate horizon compaction lowers the required threshold to $\text{AUC}^* = 1 - \frac{4}{14} \approx 0.7143$.

E.2 Mathematical Proof of Proposition 1: Sharp Affine Expected-Rank Lower Bound

Proposition 1 (Sharp Affine Expected-Rank Lower Bound). *Let* $\text{Rank}(v^*) \in \{1, 2, \ldots, M\}$ *be the integer rank of the unique gold target in a candidate pool of size* $M \geq k$*. Define top-k localization accuracy as* $p \triangleq \mathbb{P}(\text{Rank}(v^*) \leq k) = \text{Acc@}\,k$*. Then, for any distribution on* $\text{Rank}(v^*)$*:*

$$\text{Acc@}\,k \geq \max\left(0, \frac{k + 1 - \mathbb{E}[\text{Rank}(v^*)]}{k}\right). \tag{26}$$

In particular, whenever the first-moment expected-rank condition is satisfied ($\mathbb{E}[\text{Rank}(v^*)] \leq k$*), top-k localization accuracy is bounded from below by the theoretical floor:*

$$\text{Acc@}\,k \geq \frac{1}{k}. \tag{27}$$

Furthermore, the bound (26) is sharp: for any $M \geq k+1$ and any achievable mean rank $\mu \in [1, k+1]$, the two-point extremal distribution supported on $\{1, k+1\}$ achieves exact equality in (26). Notice that this proposition defines an extremal Markov support envelope: when $\mathbb{E}[\text{Rank}(v^*)] \geq k+1$, the lower bound evaluates to 0.0, and when $\mathbb{E}[\text{Rank}(v^*)] \leq k$, it establishes a floor of $1/k$ (0.20 at $k = 5$, equaling random choice). It does not serve as an operational predictor of empirical accuracy, which remains governed by Equation (23).

Proof. By definition of expectation for discrete integer-valued random variables on $\{1, \ldots, M\}$:

$$\mathbb{E}[\text{Rank}(v^*)] = \sum_{r=1}^{M} r \cdot \mathbb{P}(\text{Rank}(v^*) = r) \tag{28}$$

$$= \sum_{r=1}^{k} r \cdot \mathbb{P}(\text{Rank}(v^*) = r) + \sum_{r=k+1}^{M} r \cdot \mathbb{P}(\text{Rank}(v^*) = r). \tag{29}$$

Because $r \geq 1$ for all $r \in \{1, \ldots, k\}$, the first summation satisfies:

$$\sum_{r=1}^{k} r \cdot \mathbb{P}(\text{Rank}(v^*) = r) \geq 1 \cdot \sum_{r=1}^{k} \mathbb{P}(\text{Rank}(v^*) = r) = 1 \cdot p. \tag{30}$$

Because $r \geq k+1$ for all $r \in \{k+1, \ldots, M\}$, the second summation satisfies:

$$\sum_{r=k+1}^{M} r \cdot \mathbb{P}(\text{Rank}(v^*) = r) \geq (k+1) \cdot \sum_{r=k+1}^{M} \mathbb{P}(\text{Rank}(v^*) = r) = (k+1)(1-p). \tag{31}$$

Summing these two lower bounds establishes:

$$\mathbb{E}[\text{Rank}(v^*)] \geq p + (k+1)(1-p) = k + 1 - k \cdot p. \tag{32}$$

Rearranging terms for p:

$$k \cdot p \geq k + 1 - \mathbb{E}[\text{Rank}(v^*)] \implies p \geq \frac{k + 1 - \mathbb{E}[\text{Rank}(v^*)]}{k}. \tag{33}$$

Since probabilities are non-negative ($p \geq 0$), this establishes Equation (26). When $\mathbb{E}[\text{Rank}(v^*)] \leq k$, substituting $\mathbb{E}[\text{Rank}(v^*)] \leq k$ into Equation (26) yields:

$$p \geq \frac{k + 1 - k}{k} = \frac{1}{k}, \tag{34}$$

proving Equation (27).

To prove sharpness whenever $M \geq k+1$, consider the two-point extremal distribution supported on $\{1, k+1\} \subseteq \{1, \ldots, M\}$:

$$\mathbb{P}(\text{Rank}(v^*) = 1) = p, \qquad \mathbb{P}(\text{Rank}(v^*) = k+1) = 1 - p. \tag{35}$$

For this distribution, the expected rank is exactly:

$$\mathbb{E}[\text{Rank}(v^*)] = p \cdot 1 + (1-p)(k+1) = k + 1 - k \cdot p, \tag{36}$$

which satisfies Equation (26) with exact equality. Specifically, for any target mean $\mu \in [1, k+1]$, choosing $p = \frac{k+1-\mu}{k} \in [0, 1]$ yields $\mathbb{E}[\text{Rank}(v^*)] = \mu$ and $\text{Acc@}\,k = p = \frac{k+1-\mu}{k}$, proving that the bound cannot be improved without higher-moment constraints. ■

F Structural Edge-Type Distribution and Selective Routing Policy

F.1 Structural Edge-Type Distribution and Relational Reachability

To analyze why static call-graph expansion reaches defect targets outside lexical seeds, I audited the graph relation types connecting top-10 BM25 seeds to the 25 gold functions discovered exclusively through 1-hop AST expansion ($A_4 \setminus A_1$). The distribution reveals distinct software architectural patterns:

- **Caller Edges (`caller`, 56.0%, 14/25 tasks)**: The defect resides in an invoking entrypoint or dispatcher function that orchestrates the seed symbol. For example, in `django__django-16116`, the issue prompt describes a database migration command failure naming `makemigrations`; the actual defect resides in `handle`, which invokes the migration executor.

- **Callee Edges (`call`, 40.0%, 10/25 tasks)**: The defect resides in an internal implementation subroutine invoked by the seed. For example, in `django__django-11951`, the issue prompt describes a batch insertion constraint violation naming `bulk_create`; the bug resides in the un-named private helper `_batched_insert`.
- **Inheritance & Polymorphic Edges (`inherit`, 4.0%, 1/25 tasks)**: In `sympy__sympy-13372`, the defect resides in `evalf` (`sympy/core/evalf.py`), invoked across mathematical expression nodes.

Within 1-hop graph additions ($A_4 \setminus A_1, N = 25$), 14 tasks are callers and 10 tasks are callees of a lexical seed. Complete multi-hop reachability analysis shows that 43.7% of reachable non-seed targets require 2 hops.

F.2 Selective Routing Policy Formulation

Because defect accessibility varies markedly across issue descriptions, uniform graph expansion is suboptimal. I formulate selective routing as a contextual decision problem. Let $\mathbf{x} \in \mathbb{R}^d$ denote a feature vector extracted from the issue description q prior to candidate generation:

1. **Lexical Identifier Density**: $\rho_{\text{id}}(q) = \frac{|\,\text{Identifiers}(q)\,|}{|\,\text{Tokens}(q)\,|}$, measuring the proportion of code-like tokens (snake_case, CamelCase) in the prompt.
2. **Traceback Presence**: Binary indicator $\mathbb{I}_{\text{trace}}(q) \in \{0, 1\}$ detecting Python exception stack traces.
3. **Seed BM25 Skewness**: Skewness of the top-10 lexical score distribution $\kappa(s) = \frac{\frac{1}{10}\sum(s_i - \bar{s})^3}{\sigma_s^3}$.

A routing policy $\pi : \mathbf{x} \mapsto \{A_1, A_2, A_4\}$ selects the candidate generation strategy:

$$\pi^*(\mathbf{x}) = \begin{cases} A_1 \text{ (Seeds Only)} & \text{if } \rho_{\text{id}}(q) \geq \theta_{\text{high}} \text{ (explicit named symbol)}, \\ A_4 \text{ (Graph 1-Hop, } M = 15) & \text{if } \mathbb{I}_{\text{trace}}(q) = 1 \vee \kappa(s) > \theta_{\text{skew}}, \\ A_2 \text{ (Depth BM25)} & \text{otherwise (diffuse symptom report).} \end{cases} \tag{37}$$

Cross-validated evaluation on SWE-bench Verified demonstrates that this selective policy achieves 47.38% Acc@5 while reducing total candidate generation and model scoring compute by 31.4% compared to universal graph expansion.

G Syntactic AST Patch Replacement Tooling ($N = 30$ Synthetic Archetypes)

To protect local developer agents from syntactically invalid edits without expensive cloud retry loops, Perron provides an AST surgical replacement utility. Evaluated across 30 curated syntax failure archetypes (`results/editor_eval_results.json`) under four structural deformation regimes (clean standard, indentation trap, hunk drift, and syntax corruption), standard unified diff `git apply` suffered a 50.0% syntax error rate and 20.0% hunk rejection rate, yielding only 30.0% valid applications. In contrast, Perron's AST-directed surgical editor attained a 0.0% syntax error rate (enforced as an architectural invariant via pre-commit `ast.parse()` validation gating) and 76.7% valid patch application under structural indentation and hunk drift (with 23.3% safe rollback on corrupted syntax nodes, exact McNemar $p < .001$ vs. `git apply`).

Table 11: AST surgical editor evaluation across 30 synthetic failure archetypes. Evaluates valid patch application, syntax error rate, and rejected hunks across four deformation regimes compared to standard `git apply`.

Patch Application Engine	Valid Apply (%)	Syntax Errors (%)	Hunk Rejection (%)	Safe Rollback (%)
Standard `git apply`	30.0% (9/30)	50.0% (15/30)	20.0% (6/30)	0.0% (0/30)
Perron AST Surgical Editor	**76.7%** (23/30)	**0.0%** (0/30)	**0.0%** (0/30)	**23.3%** (7/30)

This provides compact models with an automated syntax-safety execution barrier against hallucinated indentation and truncated AST scopes, though end-to-end multi-turn benchmark resolution across full Docker environments remains an active engineering priority.

H Perron CSR SpMV Engine Architecture and Systems Benchmarks

The Perron engine is implemented with zero-copy memory-mapped CSR ingestion and compiled C/NumPy BLAS vectorization:

- **Ingestion Latency**: Loading a precomputed CSR graph via memory-mapped `numpy.load` (`mmap_mode='r'`) takes < 1.2 ms warm attach (46.97 ms cold start) across all repositories.
- **Memory Footprint**: Isolated graph array resident set size (RSS) delta is ≤ 3.0 MB, while total process peak RSS during SpMV diffusion is bounded below 144.33 MB RAM even on SymPy ($|\mathcal{V}| = 37{,}442$, $|\mathcal{E}| = 148{,}912$).
- **Throughput Comparison**: Compared to standard NetworkX PageRank (320 ms on Requests), SciPy direct sparse LU factorization via `scipy.sparse.linalg.spsolve` (SuperLU, 28.1 ms on Requests; scaling poorly on larger

graphs due to fill-in growth during LU decomposition), and standard SciPy CSR power iteration via `scipy.sparse.csr_matrix` SpMV (8.4 ms on Requests, 65.2 ms on SymPy), Perron executes optimized vectorized sparse power iteration in 1.89 ms on Requests (90 iterations, amortizing to ~ 0.021 ms per inner-loop SpMV step) and 5.10 ms on SymPy (66 iterations, ~ 0.077 ms per step; 196–529 qps; standalone single-step Python API calls incur 0.17–0.45 ms dispatch overhead). Across repositories, Perron achieves a $4.44\times$ –$12.78\times$ speedup over SciPy CSR power iteration ($4.44\times$ on Requests, $5.21\times$ on Flask, $11.49\times$ on Django, $12.78\times$ on SymPy), specifically exceeding $10\times$ on large repository graphs ($|\mathcal{V}| \geq 20{,}000$) and $> 50\times$ over NetworkX. The $12.78\times$ latency advantage over SciPy CSR power iteration on SymPy (5.10 ms vs. 65.2 ms) stems from execution mechanics rather than algorithmic divergence: SciPy's `csr_matrix.dot()` incurs Python wrapper call overhead, per-iteration intermediate dynamic memory allocations for output vectors, and repeated sparsity-structure type checks within each power step. In contrast, Perron pre-allocates contiguous zero-copy scratch arrays for the state vector and performs fused in-place vector updates directly over memory-mapped arrays, reducing per-iteration SpMV latency to 21–77 μs (corresponding to 0.517 ns per edge on SymPy, achievable only via compiled C vectorization).

Table 12: CSR SpMV diffusion engine performance benchmarks. Compares graph load latency, per-query diffusion solve time, and peak memory footprint across representative SWE-bench repositories against NetworkX, SciPy direct SuperLU sparse solver (`scipy.sparse.linalg.spsolve`), and SciPy CSR power iteration (`scipy.sparse.csr_matrix`).

Repository	Vertices $\lvert\mathcal{V}\rvert$	Edges $\lvert\mathcal{E}\rvert$	NetworkX (ms)	SciPy SuperLU (ms)	SciPy CSR (ms)	Perron CSR (ms)
psf/requests	1,482	4,120	320.4	28.1	8.4	**1.89**
pallets/flask	2,341	6,892	452.1	34.6	11.2	**2.15**
django/django	22,419	89,412	2,890.0	142.0	48.5	**4.22**
sympy/sympy	37,442	148,912	4,920.0	210.5	65.2	**5.10**

H.1 Compressed Sparse Row (CSR) Memory Layout and Vectorization

The Perron multigraph engine represents software dependency graphs using three contiguous NumPy arrays with zero-copy memory mapping:

- **Row Pointers (`indptr`)**: A 32-bit integer array of size $|\mathcal{V}| + 1$, where `indptr`$[u]$ and `indptr`$[u+1]$ demarcate the slice of outgoing edges from symbol u.
- **Column Indices (`indices`)**: A 32-bit integer array of size $|\mathcal{E}|$, storing target symbol indices for each edge.
- **Edge Weights (`data`)**: A 32-bit floating-point array of size $|\mathcal{E}|$, storing normalized transition probabilities $\bar{T}_{uv}$.

The sparse matrix-vector multiplication (SpMV) inner loop executes vectorized power iteration:

$$\boldsymbol{\pi}^{(t+1)\top} = \beta \boldsymbol{\pi}^{(t)\top} \bar{T} + (1-\beta)\mathbf{p}_0^{\top}. \tag{38}$$

Because software dependency graphs exhibit extreme sparsity (mean out-degree $\bar{d}_{\text{out}} \in [2.7, 4.0]$ across repositories, sparsity ratio $> 99.98\%$), the SpMV operation requires only $O(|\mathcal{E}|)$ floating-point operations per iteration. Executing via compiled C and NumPy BLAS vectorization over contiguous memory-mapped CSR buffers, each iteration executes in 21–77 μs (0.517 ns per edge on SymPy, or ~ 1.55 CPU cycles at 3.0 GHz). Convergence to L_1 total variation tolerance $\epsilon_{\text{tol}} = 10^{-6}$ occurs within $t \leq 60$–90 steps by the Doeblin contraction property (Appendix A.2), bounding total solve latency below 5.10 ms even on SymPy ($|\mathcal{V}| = 37{,}442$, $|\mathcal{E}| = 148{,}912$).

H.2 Cache Locality and Memory Footprint Analysis

Because the CSR representation stores outgoing edges contiguously in memory, sequential traversal during power iteration maximizes CPU L1/L2 cache line utilization. Across all 12 evaluated repositories:

- **Zero-Copy Cold Starts**: Memory-mapped attachment via `numpy.load(..., mmap_mode='r')` loads graph metadata in < 1.2 ms without allocating copy buffers.
- **Working Set Memory**: Resident set size (RSS) during active diffusion never exceeds 144.33 MB, allowing the graph engine to run continuously in background worker threads without competing for GPU VRAM.

I Sparse MoE Quantization, Physical Hardware Envelope, and Execution Telemetry

I.1 Cross-Architecture Scaling Surface and Hardware Memory Limits

To evaluate whether candidate discrimination capacity scales with model parameters, protocol `PROTOCOL-SCALING-v1` evaluates candidate discrimination across the 58 held-out Stratum C tasks where the gold target is reachable within the multi-pool candidate union ($\bigcup_i A_i$, $M_{\text{union}} \leq 87$, mean $M = 61.9$) across evaluated parameter scales of the Gemma 4 family: E2B (2B PLE dense), E4B (4B PLE dense), and 26B-A4B (3.8B active / 26B total sparse MoE).

Table 13: Gemma 4 model family execution envelope and memory requirements. Summary of active parameters, native context windows, target hardware tiers, and runtime VRAM footprints measured on Dual Tesla T4 GPUs under protocol `PROTOCOL-SCALING-v1`.

Model Variant	Params (Act/Total)	Context	Format	Runtime VRAM	Target Tier
Gemma 4 E2B	2.0B / 2.0B	128K	PLE Dense	6.7 GB	Mobile / Edge / Dual T4
Gemma 4 E4B	4.0B / 4.0B	128K	PLE Dense	9.6 GB	Consumer Laptops / Dual T4
Gemma 4 26B-A4B	3.8B / 26B	256K	Dynamic MoE	52.0 GB (FP16)	Dual T4 (Exceeded) / A100

I.2 Proportional RoPE (p-RoPE) Context Scaling and KV-Cache Memory

The Gemma 4 architecture introduces Proportional Rotary Position Embedding (p-RoPE) to sustain long-context reasoning up to 256K tokens by dynamically allocating rotational frequency scales across attention dimensions without positional degradation. For an agent processing repository context of length L tokens, key-value memory consumption scales linearly with sequence length L according to:

$$\text{Memory}_{\text{KV}}(L) = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times L \times b_{\text{precision}}, \tag{39}$$

where $b_{\text{precision}} = 2$ bytes for 16-bit precision, n_{layers} is the number of transformer layers, and $n_{\text{kv_heads}}$ is the number of key-value attention heads under Grouped Query Attention (GQA). For Gemma 4 26B-A4B ($n_{\text{layers}} = 30, n_{\text{kv_heads}} = 8, d_{\text{head}} = 128$):

$$\text{Memory}_{\text{KV}}(L) = 2 \times 30 \times 8 \times 128 \times L \times 2 = 122{,}880 \times L \text{ bytes} \approx 120 \text{ KB per token}. \tag{40}$$

At an agent context window of $L = 4{,}096$ tokens, the KV-cache requires 0.50 GB (503.3 MB) of VRAM, whereas scaling to the full 256K context window ($L = 262{,}144$) demands 32.2 GB of KV memory alone. Combined with the 52.0 GB required for unquantized 3D MoE expert weights, total memory demand reaches 52.50 GB at $L = 4$K (and 84.21 GB at $L = 256$K), exceeding the capacity of standard workstation GPUs (e.g., Dual Tesla T4 with 29.12 GB total, or single RTX 3090/4090 with 24 GB). Consequently, deploying local developer agents requires either 4-bit quantization (AWQ/GPTQ) or sliding-window attention truncation to keep memory allocations within consumer GPU budgets.

Evaluating Gemma 4 26B-A4B under protocol `PROTOCOL-SCALING-v1` isolates a practical deployment constraint for mixture-of-experts developer agents. In modern deep learning frameworks, the 128 dynamic experts are instantiated as 3D weight tensors (`[num_experts, in_features, out_features]`), which bypass standard 2D `bitsandbytes` NF4/INT8 linear layer quantization hooks without custom CUDA kernels. In unquantized 16-bit precision, expert parameters require 52.0 GB of runtime VRAM (44.0 GB compressed disk storage), physically exceeding dual Tesla T4 capacity (29.12 GB). Attempting runtime CPU or disk offloading degrades inference speed below interactive usability. Consequently, while compact dense models ($\leq$ 4B parameters) operate smoothly within local GPU budgets ($\leq$ 9.6 GB), deploying high-capacity sparse MoE models for local developer agents requires ahead-of-time AWQ or GPTQ quantization formats with native 3D tensor support.

J Qualitative Failure Traces and Defect Localization Case Studies

This appendix examines representative failure modes and qualitative traces from SWE-bench Verified tasks to illustrate the interplay between static AST graph topology, distractor dilution, and test-time reasoning.

J.1 Case Study 1: Graph Expansion and Dispatch Reasoning in Django

In task `django__django-11951` (SWE-bench Verified), the issue report describes a batch insertion failure where custom batch sizes are ignored:

> *"bulk_create ignores batch_size parameter when determining maximum batch sizes for database backends. Setting a custom batch_size still attempts to insert all objects in batches capped by max_batch_size instead of min(batch_size, max_batch_size)."*

Top-10 BM25 lexical search identifies high-level entrypoints in `django/db/models/query.py` (`bulk_create`, `create`), allocating top rank to `bulk_create` due to prompt keyword overlap. However, the defect arithmetic resides in the un-named internal helper `_batched_insert`. 1-hop AST expansion from `bulk_create` traverses the internal invocation edge to inject `_batched_insert` into candidate pool A_4. Under direct scoring without reasoning, the model retains `bulk_create` at rank 1 due to prompt keyword density. When test-time deliberation is enabled, intermediate reasoning successfully reflects on method delegation, identifying that `bulk_create` forwards row chunking to `_batched_insert`, thereby promoting the helper to Top-1.

J.2 Case Study 2: Distractor Dilution via High-Degree Utility Sinks

In task `django__django-11019`, the issue report concerns a widget media merging regression in formsets.

- **Seed Retrieval**: Lexical seeds identify `forms/widgets.py` and the `Media` class.
- **Standard PPR Collapse**: Standard Personalized PageRank without specificity damping funnels over 45% of its diffusion mass into ubiquitous utility functions and scale-free hubs (e.g., `BlockNode.super`, in-degree 1,362; `setup`, in-degree 926), displacing domain-specific candidates.
- **Specificity Damping Protection**: Perron's decoupled specificity ratio ($\gamma = 0.70$) suppresses these generic hubs (HSI@10 = 99.5%), successfully retaining the target `Media.__add__` in the top-5 candidate pool.

J.3 Case Study 3: Syntactic AST Scope Failure and Atomic Rollback

In Synthetic Archetype #14 (Indentation Trap from the AST Patch Deformation Suite, $N = 30$, `results/editor_eval_results.json`), an automated edit introduces an indentation mismatch into a multi-line decorator block:

```
<<<<<<<
    @property
    def infer(self):
        return self._infer()
=======
    @property
def infer(self):
    return self._infer()
>>>>>>>
```

Standard `git apply` applies the patch with line offset drift, producing an `IndentationError` at runtime. In contrast, Perron's AST surgical editor detects that the patch breaks parent class AST scope before committing changes to disk, executes an atomic rollback, and emits a structured error receipt.

J.4 Case Study 4: Cross-File Import Boundary Disconnect

In task `pytest-dev__pytest-7571`, the defect requires restoring the log level inside `src/_pytest/logging.py` after a test utilizing the `caplog` fixture finishes. Lexical search identifies `src/_pytest/terminal.py`. Because fixture registrations and cleanup hooks execute dynamically at pytest session runtime via plugin dispatch hooks rather than static AST `import` or `call` statements, static graph diffusion fails to traverse the fixture lifecycle hook edge (0-hop static connectivity), illustrating the fundamental boundary condition where dynamic reflection decouples from static AST multigraphs.

K Scoring Prompt Templates, Verbalization Schemas, and Decoding Specifications

This appendix documents the complete prompt templates, verbalization schemas, and decoding configurations used across all experimental protocols.

K.1 Pointwise Candidate Reranking Prompt (Protocol Lock 9abe405)

In confirmatory pointwise candidate reranking (Sections 4.1–4.2, evaluating candidate arms B0–A6 across $N = 344$ tasks), each candidate symbol is formatted as an AST skeleton breadcrumb and evaluated in isolation via a single forward pass without generation:

```
<start_of_turn>user
Below is a GitHub issue for a Python repository and one candidate function from that repository.

<issue>
{issue_text}
</issue>

<candidate>
{candidate_breadcrumb}
</candidate>

Answer Yes only if the code of this specific function must be changed
to fix the issue. Answer No if the function is merely related to the issue,
calls the buggy code, or is called by it.
Answer with exactly one word: Yes or No.<end_of_turn>
<start_of_turn>model
```

Scoring extracts final-position unnormalized logits z for affirmative tokens $\mathcal{Y} = \{\texttt{Yes}, \texttt{yes}\}$ and negative tokens $\mathcal{N} = \{\texttt{No}, \texttt{no}\}$ upcast to FP32, computing candidate log-odds:

$$s(t, v) = \text{logsumexp}_{y \in \mathcal{Y}}(z_y) - \text{logsumexp}_{n \in \mathcal{N}}(z_n). \tag{41}$$

K.2 Listwise Direct Readout and Test-Time Reasoning Prompts (Protocol Lock c563441)

In Stratum C factorial evaluation (Section 4.4, $N = 131$ tasks), candidate pools ($M \in \{15, 25\}$) are packed into a single prompt under $K \leq 3{,}480$ tokens. Under **Direct Readout** (C_0, $M = 15/25$), the prompt instructs the model to emit a top-5 ranking:

```
You are an expert developer resolving a software issue in a codebase.

### Issue Description:
{issue_text}

### Candidate Functions ({M} candidates retrieved):
[Candidate ID: 1]
# File: ...
def func1(...): pass

[Candidate ID: 2]
...

### Task:
Identify the primary root-cause location that must be modified to fix the issue.
Output your top-5 ranking by Candidate ID in descending order of likelihood.
Format your ranking strictly in tags:
<ranking>[ID1, ID2, ID3, ID4, ID5]</ranking>.
```

Under **Test-Time Reasoning** (C_0+think, C_3), intermediate deliberation is enabled via the chat template:

```
### Task:
Identify the primary root-cause location that must be modified to fix the issue.
First, reflect on candidate functionality and root-cause likelihood
inside <thought>...</thought> tags.
Then, output your top-5 ranking by Candidate ID in descending order of likelihood.
Format your ranking strictly in tags:
<ranking>[ID1, ID2, ID3, ID4, ID5]</ranking>.
```

For provenance conditions C_2 and C_3, candidate stubs prepend explicit relation comments: `# Provenance: Called by <seed>` or `# Provenance: Calls <seed>`.

K.3 Decoding Parameters and Truncation Backfilling

All listwise evaluations execute under deterministic greedy decoding ($T = 0.0$, top-$p = 1.0$) with generation capped at 180 tokens. In reasoning conditions, generating multi-step deliberation occasionally causes generation truncation before 5 complete candidate IDs are emitted. In compliance with protocol specification `PROTOCOL-CAUSAL-v1`, missing rank positions (ranks 2–5) are backfilled deterministically by preserving unranked candidates in their original prompt retrieval order.

L Open-Source Ecosystem, Artifact Hashes, and Monograph Specification

To ensure independent auditability and empirical reproducibility for the post-competition winner review, Perron is released as an open resource ecosystem under Apache 2.0:

- **`scores.parquet`**: 19,298 scored candidate evaluations across all 344 SWE-bench Verified tasks under Apache 2.0 (https://github.com/yvliet/perron). SHA-256: `f75bbed78c5d13385aa337d6f12bc0eead0a2c4866326698f86189e9abb9609f`.
- **Evaluation Protocol Specification**: `benchmarks/protocol/gemma4_protocol_v1.yaml` (commit `a13ab34`) SHA-256: `96af1621a27b48a99666a8fcef626ecd2415fd094fc6b6803fce567c38d927ce`.
- **Execution Environment Candidate Lock**: `benchmarks/protocol/gemma4_lock_v1.json` (commit `9abe405`) SHA-256: `281508f0f0a35e03cbbfd5b4ec3d69913e3f0c995552252cd68a33e95ec9dfbc` (CRLF: `dbbf292032b2fdff39692476b3d557dddd437a176b745f4ae63ca34fab254712`).
- **Benchmark Task Split Lock**: `data/swebench_lite_split.json` SHA-256: `95c7ef3ca69214f5ac9f6bcba20e2531596ed260c2cbe181d2df3a56873d259b` (CRLF: `18a947a0e8a7941231ab15f11c1eee335adc58f8d03cbce742637452c9a23418`).
- **Classifier-vs-Human Validation Audit**: `results/strata_human_validation.json` ($N = 100$, Cohen's $\kappa = 0.918$) SHA-256: `f08b83d3976e46e5077df7deb8843f18bd0af1392298dd7f2a80cb61eabefcdd`.
- **`perron-core` v0.3.0**: High-throughput Python library providing zero-copy memory-mapped CSR ingestion, prior normalization, and AST breadcrumb packing on PyPI (https://pypi.org/project/perron-core/).
- **`perron-mcp`**: JSON-RPC stdio server enabling any MCP-compliant agent or IDE (VS Code, Claude Desktop, Cursor) to execute spectral diffusion and provenance extraction locally via `npx` or `python -m perron.mcp`.

- **High-Level Python API**: Drop-in interface (`diffuse_with_provenance`) enabling developer agent frameworks (Aider, Agentless, SWE-agent) to replace static repository tag maps with high-throughput CSR diffusion and caller-callee provenance.
- `swebench-codegraphs`: Precomputed CSR graphs for all 12 canonical SWE-bench repositories (covering both Lite and Verified tasks, built with `scripts/export_swebench_graphs.py`).
- `swebench-strata-annotations`: 800 lexical accessibility annotations across SWE-bench (300 Lite + 500 Verified tasks; 344 callable Verified benchmark tasks, CC0-1.0).
- **Kaggle Demonstration Workstation**: Interactive reproduction workstation executing on free Dual Tesla T4 GPUs, validating 344-task context packing and full 2×2 factorial causal evaluation at https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation.
- `perron-gemma4-e2b-lora`: Fine-tuned parameter-efficient LoRA adapter ($r = 16$) optimizing provenance-conditioned candidate discrimination on Hugging Face (https://huggingface.co/yvliet/perron-gemma4-e2b-lora).

M Datasheet for Datasets: SWE-bench Strata and Repository Multigraphs

Following the documentation standard of Gebru et al. [8], this datasheet provides formal documentation for the open research assets released with Perron, covering task stratification annotations, repository-level AST multigraphs, and scored model evaluation telemetry.

M.1 Motivation

Purpose: The dataset was constructed to evaluate the empirical boundaries of static graph expansion, distractor dilution, and test-time reasoning in repository-level code intelligence, addressing the lack of structural task stratification in existing software engineering benchmarks. **Author and Funding**: Sultan Haikal (GitHub: `@yvliet`). The dataset was developed independently for the Google DeepMind Gemma 4 Developer Agent competition (Track 2: Research Monograph and Resource Track). Model inference was executed on Kaggle dual Nvidia Tesla T4 GPU environments.

M.2 Composition

The dataset comprises three primary components:

1. **SWE-bench Strata Annotations** (`data/swebench_strata_annotations.jsonl`): 800 evaluation records across 707 unique task instances (300 in SWE-bench Lite, 500 in SWE-bench Verified, with 93 overlapping pairs). Each instance is classified into Stratum A (`A_named`), Stratum B (`B_trace`), or Stratum C (`C_unnamed`). The 500 SWE-bench Verified tasks follow a strict CONSORT partition: 344 callable evaluated tasks, 93 exploratory Lite overlap, 56 file-preimage drift, and 7 module-level non-callable edits ($344 + 93 + 56 + 7 = 500$).
2. **Repository AST Multigraphs (Two-Tier Design)**:
   - **Per-Task Base-Commit Multigraphs** (`data_release/graphs/`, $N = 300$ Lite tasks): CSR matrices and symbol tables extracted from checkouts at each task's immutable git base commit for community benchmarking.
   - **Single-Snapshot Repository Multigraphs** (`artifacts/swebench_graphs/`, 12 repositories): monolithic CSR graphs (109,240 nodes, 341,892 directed edges) indexed at a frozen snapshot to benchmark static offline caching on SWE-bench Verified ($N = 344$).
3. **Scored Model Telemetry** (`scores.parquet`, `gemma4_e4b_scores.parquet`): 19,298 candidate evaluations from Gemma 4 E2B and 8,113 candidate evaluations from Gemma 4 E4B across SWE-bench Verified.

M.3 Collection Process

For SWE-bench Lite ($N = 300$, `data_release/graphs/`), repository trees were checked out at each task's exact historical git `base_commit` and parsed via Python's standard `ast` module. For the confirmatory SWE-bench Verified evaluation ($N = 344$, `artifacts/swebench_graphs/`), 12 repositories were indexed from a single frozen release snapshot to evaluate static offline agent caching without per-task re-indexing. Tasks whose target files were absent at the frozen snapshot commit ($N = 56$, `FILE_ABSENT_PREIMAGE_DRIFT`) were audited via historical `base_commit` checkouts, confirming 100% CPython 3 AST parseability (68 files, 3,827 symbols) and proving that exclusions were an operational artifact of single-snapshot caching rather than parser failure. Strata classifications were assigned via deterministic regular expression matching with strict identifier boundaries (`perron/analysis/strata.py`). Telemetry was captured during offline inference in FP16 precision.

M.4 Preprocessing, Cleaning, and Classifier-vs-Human Validation Audit

File paths were normalized to POSIX standards. Missing distances are encoded as JSON null in compliance with RFC 8259. To verify that the deterministic regex heuristic represents genuine linguistic ground truth, an independent human validation

audit was conducted on a stratified random sample of $N = 100$ tasks ($N_A = 45, N_B = 15, N_C = 40$, seed=42) against raw GitHub issue statements without access to regex tags. The audit demonstrated an observed concordance of 95.0% (95/100 tasks) and an empirical Cohen's $\kappa = 0.918$ (95% CI $[0.849, 0.988]$, chance agreement $p_e = 38.8\%$). Under Landis and Koch criteria, $\kappa = 0.918$ reflects almost perfect agreement, confirming that the automated heuristic reliably mirrors human expert classification.

M.5 Uses

The dataset is intended for benchmarking repository-level code search, graph neural networks for program analysis, and test-time reasoning architectures. Researchers should preserve the pre-specified confirmatory split ($N = 344$ callable Verified) to prevent selection conditioning.

M.6 Distribution and Maintenance

The dataset and code are released under the Apache 2.0 license at `https://github.com/yvliet/perron` and archived on Zenodo and Hugging Face. The resource is maintained by Sultan Haikal (GitHub: `@yvliet`, email: `yvliet@users.noreply.github.com`).